

Software Testing



Software Testing

- What is Testing?

Many people understand many definitions of testing :

1. Testing is the process of demonstrating that errors are not present.
2. The purpose of testing is to show that a program performs its intended functions correctly.
3. Testing is the process of establishing confidence that a program does what it is supposed to do.

These definitions are incorrect.

Software Testing

A more appropriate definition is:

“Testing is the process of executing a program with the intent of finding errors.”

Software Testing

- Why should We Test ?

Although software testing is itself an expensive activity, yet launching of software without testing may lead to cost potentially much higher than that of testing, specially in systems where human safety is involved.

In the software life cycle the earlier the errors are discovered and removed, the lower is the cost of their removal.

Software Testing

- Who should Do the Testing ?
 - o Testing requires the developers to find errors from their software.
 - o It is difficult for software developer to point out errors from own creations.
 - o Many organisations have made a distinction between development and testing phase by making different people responsible for each phase.

Software Testing

- What should We Test ?

We should test the program's responses to every possible input. It means, we should test for all valid and invalid inputs.

Suppose a program requires two 8 bit integers as inputs. Total possible combinations are $2^8 \times 2^8$. If only one second is required to execute one set of inputs, it may take 18 hours to test all combinations.

Practically, inputs are more than two and size is also more than 8 bits. We have also not considered invalid inputs where so many combinations are possible. Hence, complete testing is just not possible, although, we may wish to do so.

Software Testing

Some Terminologies

➤ Error, Mistake, Bug, Fault and Failure

People make **errors**. A good synonym is **mistake**. This may be a syntax error or misunderstanding of specifications. Sometimes, there are logical errors.

When developers make mistakes while coding, we call these mistakes “**bugs**”.

A **fault** is the representation of an error, where representation is the mode of expression, such as narrative text, data flow diagrams, ER diagrams, source code etc. Defect is a good synonym for fault.

(Or) A fault is a condition that causes a system to fail in performing its required functions.

A **failure** occurs when a fault executes. A failure is the inability of a system or component to perform a required function.

Software Testing

➤ Test Scenario: Test, Test Case and Test Suite

Test scenario: What to be tested. **Test** and **Test case** terms are used interchangeably. Test case is a test condition which is executed for expected O/P with predefined set of steps with known inputs. (or) Test case describes an input description and an expected output description.

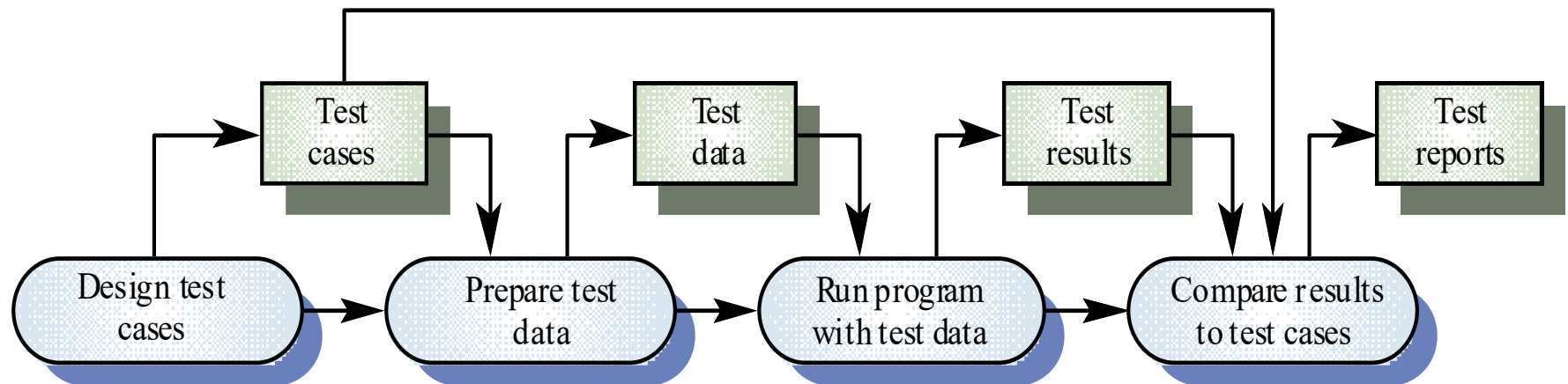
Test Case ID	
Section-I (Before Execution)	Section-II (After Execution)
Purpose :	Execution History:
Pre condition: (If any)	Result:
Inputs:	If fails, any possible reason (Optional);
Expected Outputs:	Any other observation:
Post conditions:	Any suggestion:
Written by:	Run by:
Date:	Date:

Fig. 2: Test case template

The set of test cases is called a **test suite**. Hence any combination of test cases may generate a test suite.

JIRA, Mantis are examples of software to create test cases.

The defect testing process



Software Testing

➤ Verification and Validation

Verification - typically involves reviews and meeting to evaluate documents, plans, code, requirements, and specifications. This can be done with checklists, issues lists, walkthroughs, and inspection meeting.

Validation - typically involves actual testing and takes place after verifications are completed.

Testing= Verification+Validation

Software Testing

➤ Alpha, Beta and Acceptance Testing

The term **Acceptance Testing** is used when the software is developed for a specific customer. A series of tests are conducted to enable the customer to validate all requirements. These tests are conducted by the end user / customer and may range from adhoc tests to well planned systematic series of tests.

Alpha Tests This is a form of internal acceptance testing performed mainly by the in-house software QA and testing teams. Alpha testing is the last testing done by the test teams at the development site after the acceptance testing and before releasing the software for beta test.

Alpha testing can also be done by the potential users or customers of the application. But still, this is a form of in-house acceptance testing. These tests are conducted in a controlled environment.

Software Testing

Beta Tests are conducted by the customers / end users at their sites. Unlike alpha testing, developer is not present here. Beta testing is conducted in a real environment that cannot be controlled by the developer. Three types of beta testing are:

Traditional Beta testing: Product is distributed to the target market, and related data is gathered in all aspects. This data can be used for Product improvement.

Public Beta Testing: Product is publicly released to the outside world via online channels and data can be gathered from anyone. Based on feedback, product improvements can be done. For example, Microsoft conducted the largest of all Beta Tests for its OS -- Windows 8 before officially releasing it.

Technical Beta Testing: Product is released to the internal group of an organization and gathers feedback/data from the employees of the organization.

Software Testing

Testing in the large Vs Testing in the small:

Software product are normally tested first at the individual component (or unit) level. This is referred to as testing in the small (**unit testing**).

After testing all the components individually, the components are slowly integrated and tested at each level of integration (**integration testing**).

Finally, the fully integrated system is tested (called **system testing**).

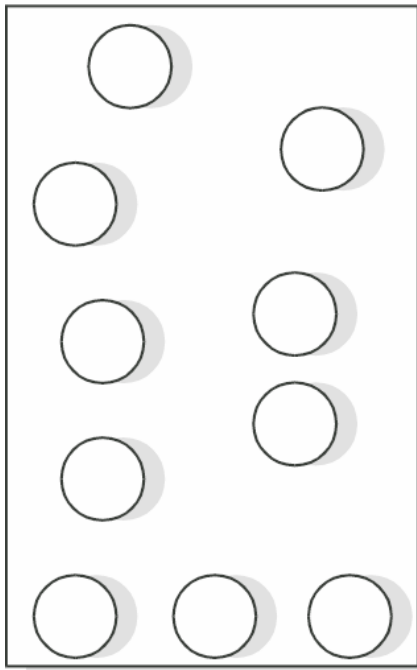
Integration and system testing are known as testing in the large.

Software Testing

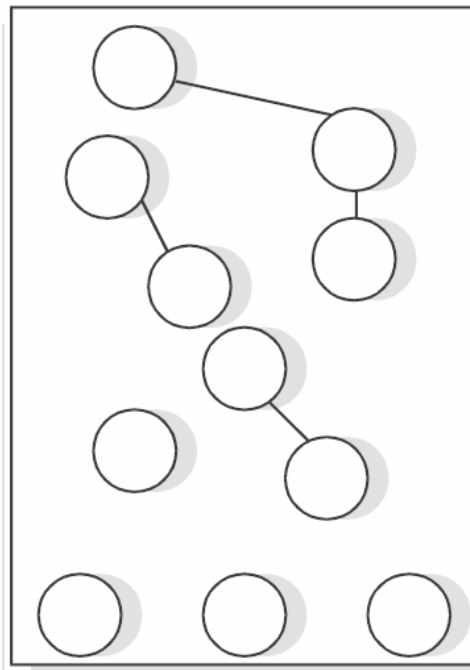
Levels of Testing

There are 3 levels of testing:

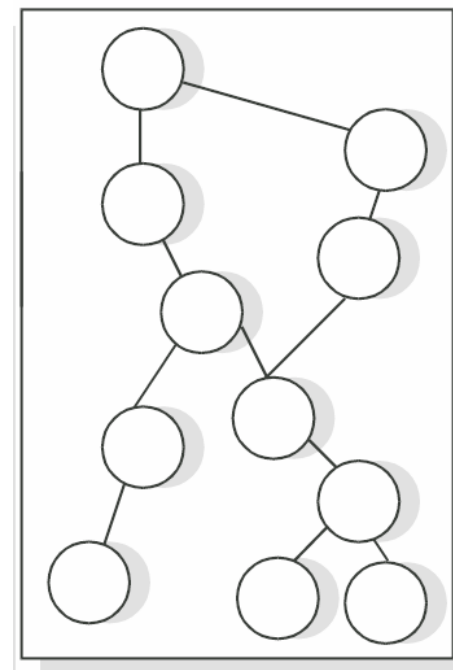
- i. Unit Testing
- ii. Integration Testing
- iii. System Testing



UNIT TESTING



INTEGRATION TESTING



SYSTEM TESTING

Software Testing

Unit testing

Unit testing is the process of taking a module and running it in isolation from the rest of the software product by using prepared test cases and comparing the actual results with the results predicted by the specification and design of the model.

- The most ‘micro’ scale of testing.
- Tests done on particular functions or code modules.
- Requires knowledge of the internal program design and code.
- Done by Programmers (not by testers).

There are many reasons in support of unit testing:

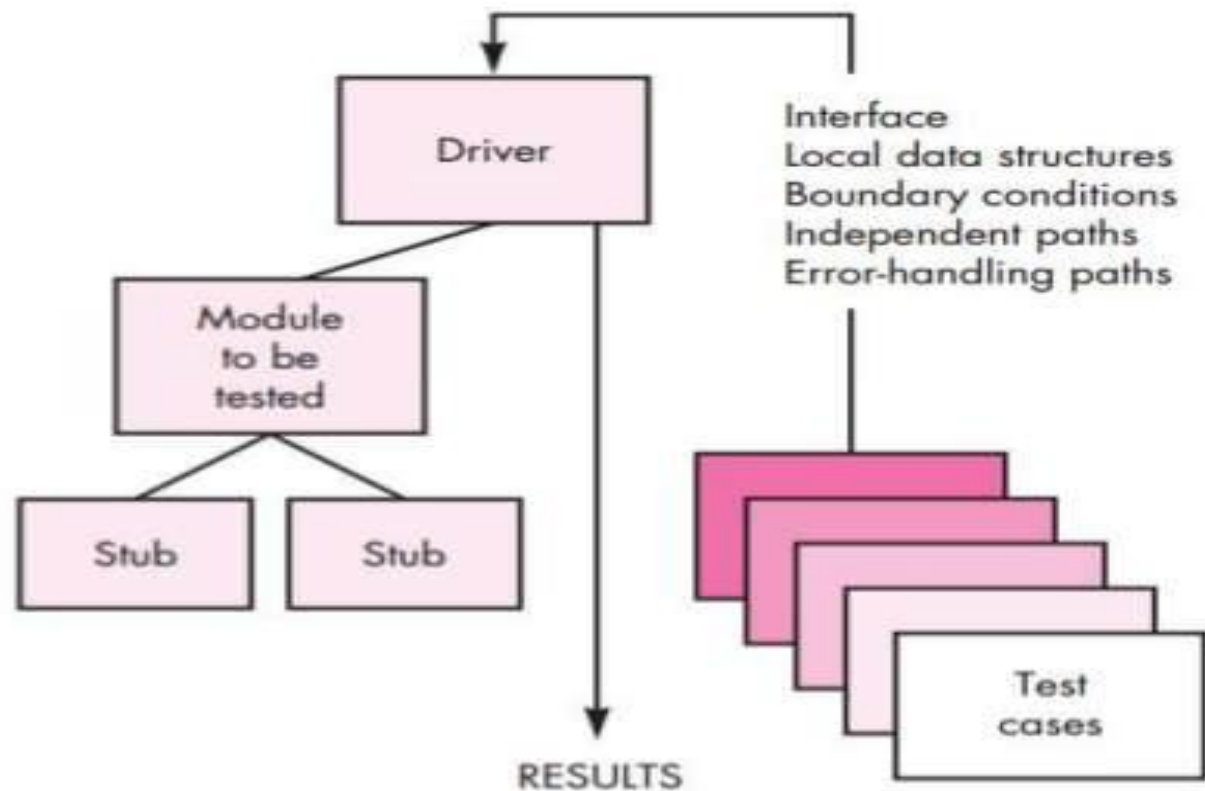
1. The size of a single module is small enough that we can locate an error fairly and easily.
2. Confusing interactions of multiple errors in different parts of software are eliminated.

Objectives	• To test the function of a program or unit of code such as a program or module • To test internal logic • To verify internal design • To test path & conditions coverage • To test exception conditions & error handling
When	• After modules are coded
Input	• Internal Application Design • Master Test Plan • Unit Test Plan
Output	• Unit Test Report

Who	•Developer
Methods	•White Box testing techniques
Tools	•Debug •Re-structure •Code Analyzers •Path/statement coverage tools

- There are problems associated with testing a module in isolation. How do we run a module without anything call it; to be called by it? One approach is to construct an appropriate **driver routine** to call it and, **simple stubs** to be called by it and to insert output statements in it.

Unit-test environment



14

- Stubs and drivers are designed to provide the complete environment for a module to be tested.
- Stubs serve to replace modules that are subordinate to (called by) the module to be tested.
- A driver is nothing more than a “main program” that accepts test case data, passes such data to the component (to be tested) and prints relevant results.

Integration Testing

- Integration testing determine the interface between modules.
- Testing of combined parts of an application to determine their functional correctness.
- During Integration testing, different modules of a system are integrated in a planned manner using an integrated plan. The integration plan specifies the steps and the order in which modules are combined to realize the full system. After each integration step, the partially integrated system is tested.

Objectives	• To technically verify proper interfacing between modules, and within sub-systems
When	• After modules are unit tested
Input	• Internal & External Application Design • Master Test Plan • Integration Test Plan
Output	• Integration Test report

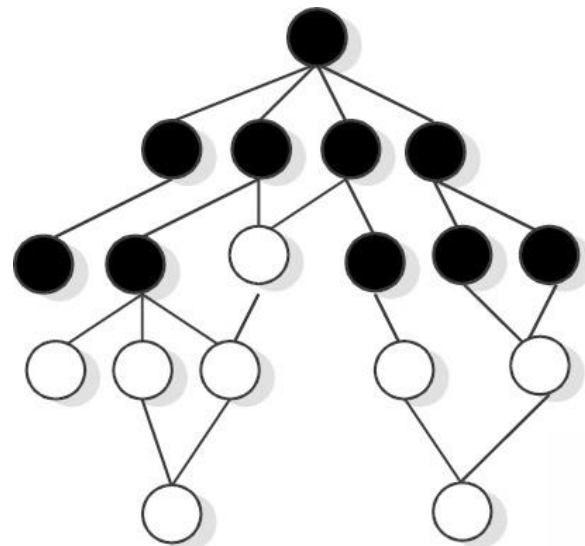
Who	•Development Team and Users
Methods	•Problem / Configuration Management
Tools	•Recommended set of tools

Why Integration Testing Is Necessary

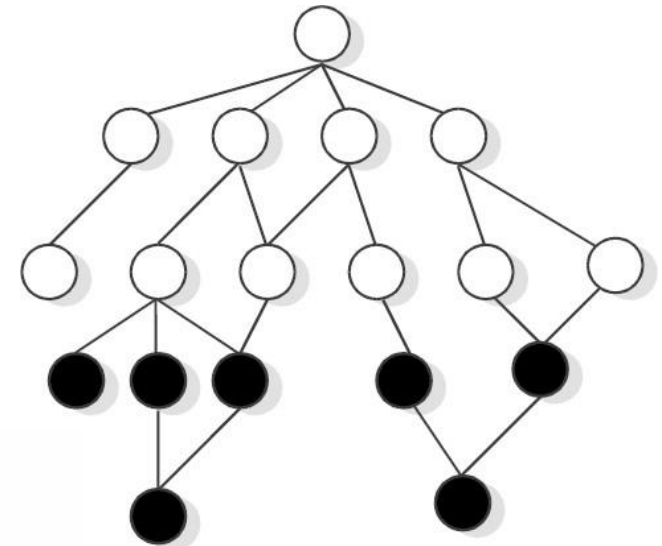
- One module can have an adverse effect on another
- Sub-functions, when combined, may not produce the desired major function
- Interfacing errors not detected in unit testing may appear
- Timing problems (in real-time systems) are not detectable by unit testing
- Resource contention problems are not detectable by unit testing

Top down and bottom approaches are used to develop the test plan in integration testing.

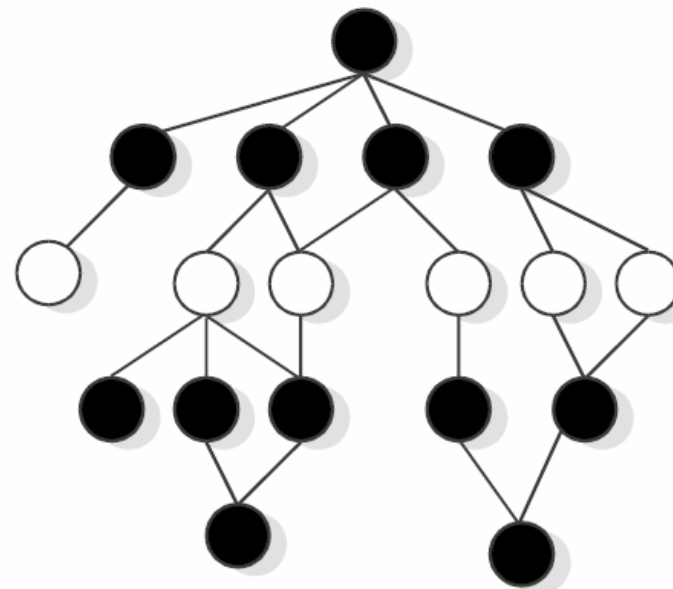
Software Testing



Top-down integration



Bottom-up integration



Sandwich integration

Fig. 30 : Three different integration approaches

Top-down Integration:

Is an incremental approach to construction of the software architecture. Modules are integrated by moving downward through the control hierarchy, beginning with the main control module (main program).

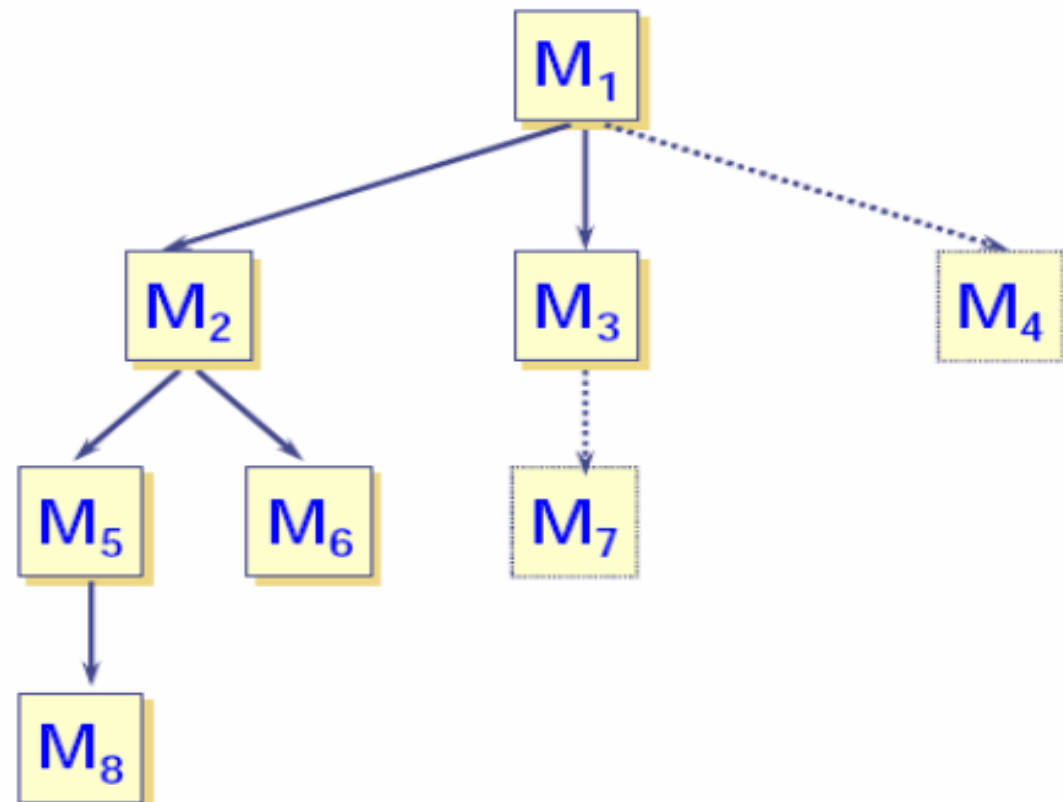
Module subordinate to the main control module are incorporated into the structure in either depth first or breadth first.

⇒ Top-Down Testing with Depth-First

- ❑ M1, M2, M5, M8
- ❑ M6
- ❑ M3, M7
- ❑ M4

⇒ Top-Down Testing with Breath-First

- ❑ M1
- ❑ M2, M3, M4
- ❑ M5, M6, M7
- ❑ M8

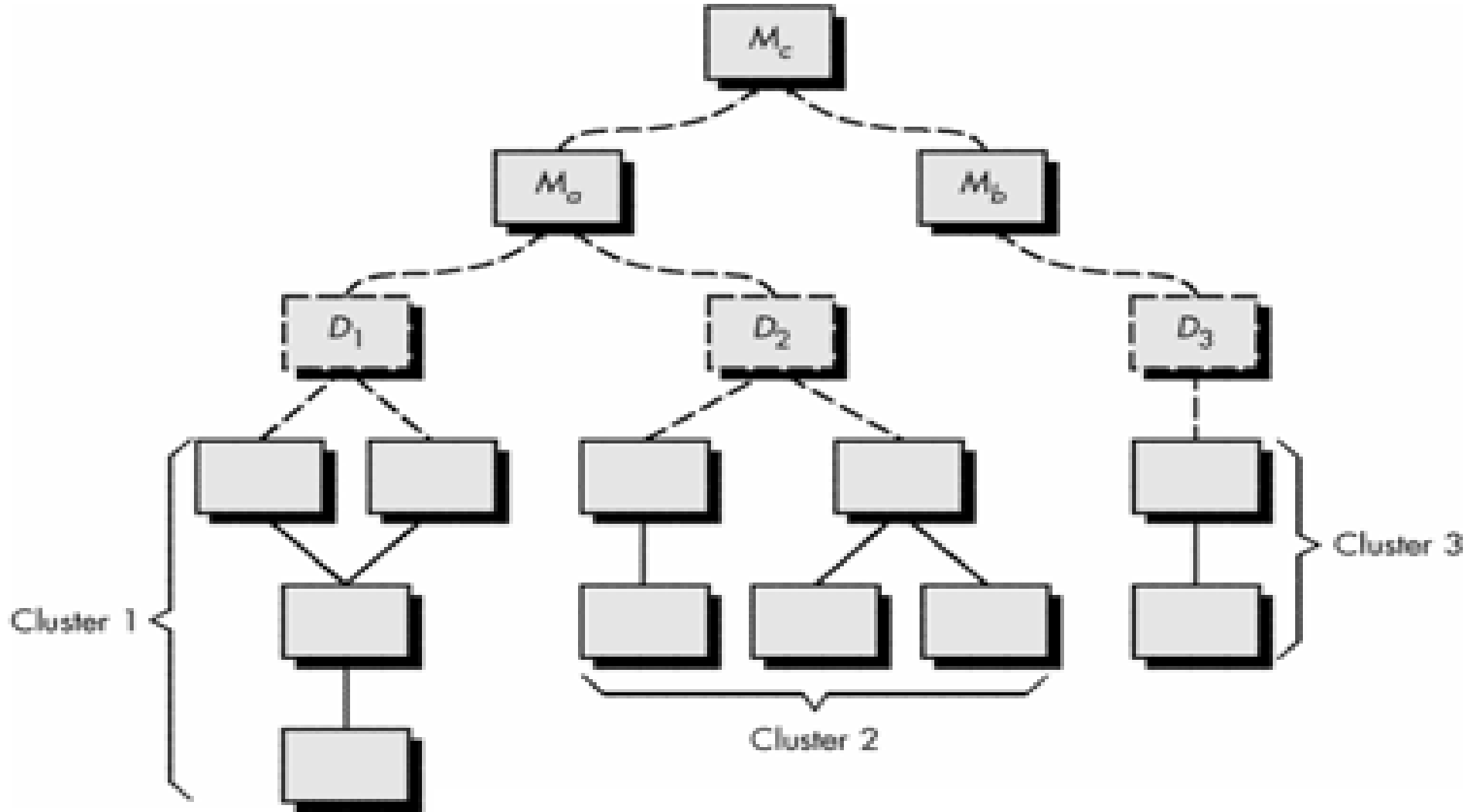


Bottom-Up Integration:

As its name implies, begins construction and testing with atomic module (i.e component at lowest level). Some steps are in it:

- Low level components are combined into clusters (some times called builds)
- A driver is written to coordinate test case input and output.
- Cluster is tested.
- Drivers are removed and clusters are combined moving upward.

Bottom-Up Integration:



Components are combined to form clusters 1,2,3. Each of the clusters is tested using a driver. Components in clusters 1 and 2 are subordinate to M_a . Drivers D_1 and D_2 are removed and clusters are interfaced directly to M_a . Similarly driver D_3 for cluster 3 is removed and integrated with M_b . Both M_a and M_b are combined integrated with M_c .

➤ Regression Testing

Each time a new module is added as part of integration testing, the software changes. New data flow paths are established, new control logic is invoked. These changes may cause of problem. Then regression testing is required.

“Regression Testing is nothing but a full or partial selection of already executed test cases which are re-executed to ensure that changes have not propagated unintended side effects”.

Regression testing may be conducted manually or using automated tools. Manually, by re-executing a subset of all test cases. If your software undergoes frequent changes, regression testing costs will escalate. In such cases, Manual execution of test cases increases test execution time as well as costs. Automation of regression test cases is the smart choice in such cases.

Following are the most important tools used for both functional and regression testing:

Selenium: This is an open source tool used for automating web applications. Selenium can be used for browser-based regression testing.

Quick Test Professional (QTP): HP Quick Test Professional is automated software designed to automate functional and regression test cases. It uses VBScript language for automation. It is a Data-driven, Keyword based tool.

➤ Smoke Testing:

Smoke testing, also known as “**Build Verification Testing**” or “**Confidence Testing**”. Smoke testing covers most of the major functions of the software but none of them in depth. The result of this test is used to decide whether to proceed with further testing. If the smoke test passes, go ahead with further testing. If it fails, halt further tests and ask for a new build with the required fixes.

Smoke test helps in exposing integration and major problems early in the cycle. It can be conducted on both newly created software and enhanced software. Smoke test is performed manually or with the help of automation tools/scripts. If builds are prepared frequently, it is best to automate smoke testing.

Note: A build (cluster) includes all data files, libraries, reusable modules, and engineered components.

System Testing

System Testing are designed to validate a fully developed system to assure that it meets its requirements.

Objectives	• To verify that the system components perform control functions • To perform inter-system test • To demonstrate that the system performs both functionally and operationally as specified • To perform appropriate types of tests relating to Transaction Flow, Installation, Reliability, Regression etc.
When	• After Integration Testing
Input	• Detailed Requirements & External Application Design • Master Test Plan • System Test Plan
Output	• System Test Report
Who	• Development Team and Users

Actually a system testing is a series of different tests as:

- Recovery Testing
- Security Testing
- Stress Testing
- Functional Testing
- Performance Testing

Recovery testing:

Testing how well a system recovers from crashes, HW failures or other problems.

Security Testing:

is a type of software testing that intends to uncover vulnerabilities of the system and determine that its data and resources are protected from possible intruders

Stress Testing:

Testing under unusually heavy loads, heavy repetition of certain actions or inputs, input of large numerical values, large complex queries to a database etc.

Term often used interchangeably with 'load' and 'performance' testing.

Performance testing:

Testing how well an application complies to performance requirements.

Functional Testing: is a type of software testing whereby the system is tested against the functional requirements/specifications.

Other Testing Names

Sanity testing:

After receiving a Software build with the minor issues fixes in code or functionality, Sanity testing is carry out to check whether the bugs reported in previous build are fixed & there is regression introduced due to these fixes i.e. not breaking any previously working functionality. The main aim of Sanity testing to check the planned functionality is working as expected. Instead of doing whole regression testing the Sanity testing is perform.

Load testing:

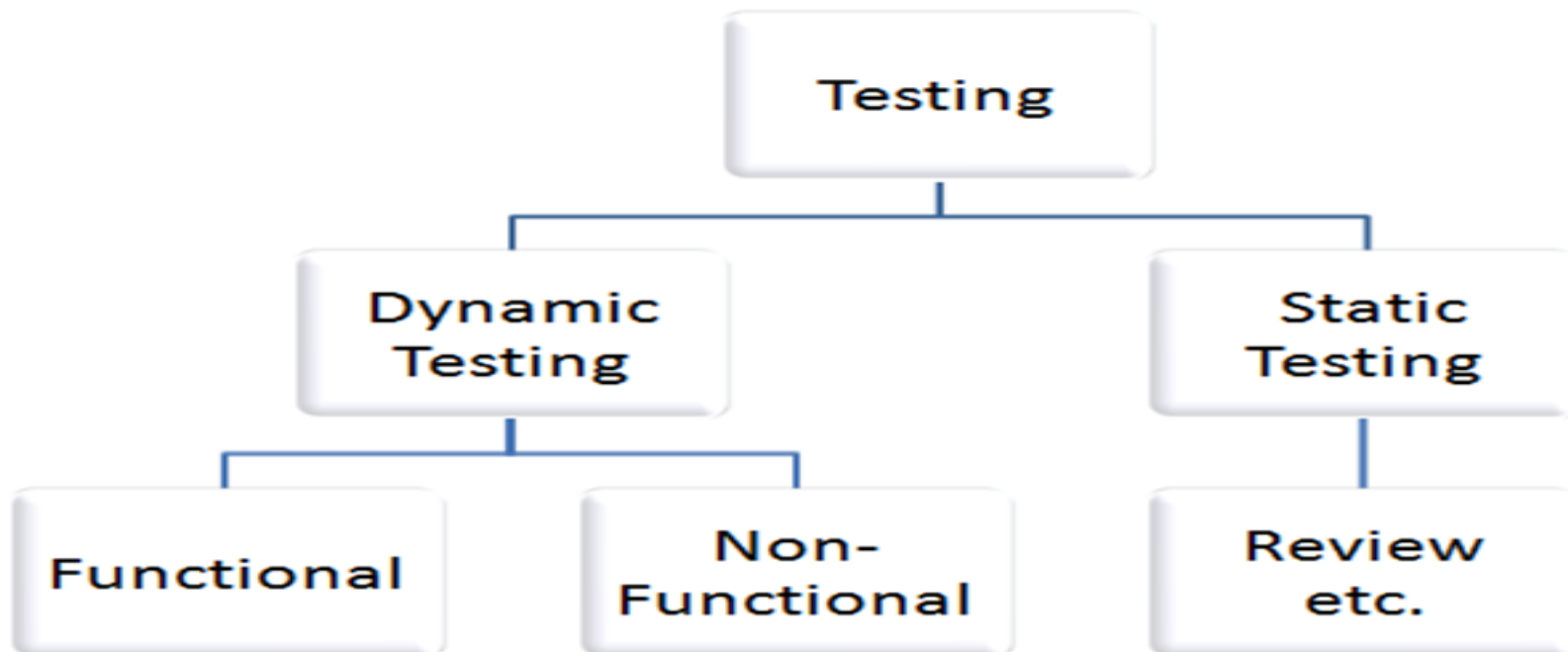
- Testing an application under heavy loads.
- Eg. Testing of a web site under a range of loads to determine, when the system response time degraded or fails.

Exploratory testing / ad-hoc testing:

Informal SW test that is not based on formal test plans or test cases; testers will be learning the SW in totality as they test it.

Testing Approaches

- Functional Testing (Black Box Testing)
- Structural Testing (White Box Testing)



Software Testing

Functional Testing

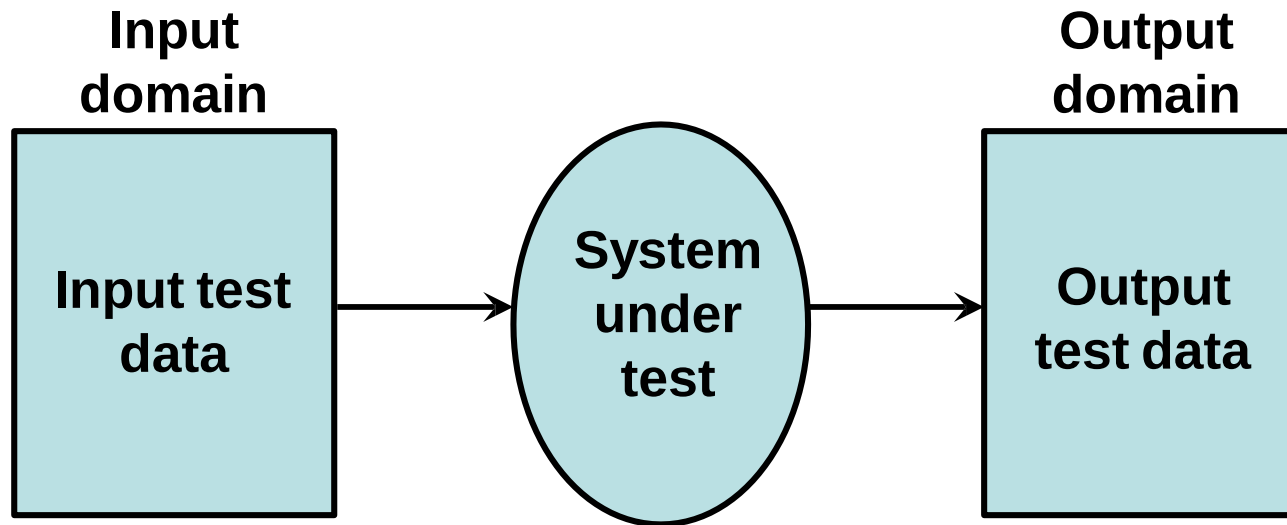


Fig. 3: Black box testing

Software Testing

Boundary Value Analysis: Boundary conditions means an input value may be on the boundary, just below the boundary or just above the boundary. Suppose we have an input variable x with a range from 1 to 100. The boundary values are 1,2,99 and 100. Consider a program with two input variables x and y . These input variables have specified boundaries as:

$$a \leq x \leq b$$

$$c \leq y \leq d$$

Input domain

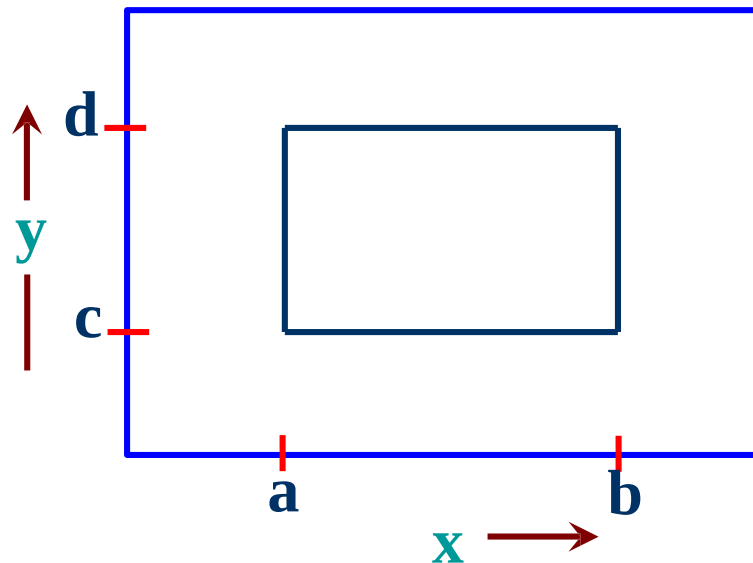


Fig.4: Input domain for program having two input variables

Software Testing

The basic idea of boundary value analysis is to use input variables at their minimum, just above minimum, a nominal value, just below their maximum, and at their maximum.

Here, we have an assumption of reliability theory known as “single fault” assumption. This says that failures are rarely the results of the simultaneous occurrence of two (or more) faults. Thus, boundary value analysis test cases are obtained by holding the values of all but one variable at their nominal values and letting that variables assume its extreme values.

Software Testing

The boundary value analysis test cases for our program with two inputs variables (x and y) that may have any value from 100 to 300 are: (200,100), (200,101), (200,200), (200,299), (200,300), (100,200), (101,200), (299,200) and (300,200). This input domain is shown in Fig. 5. Each dot represent a test case and inner rectangle is the domain of legitimate inputs. Thus, for a program of n variables, boundary value analysis yield $4n + 1$ test cases.

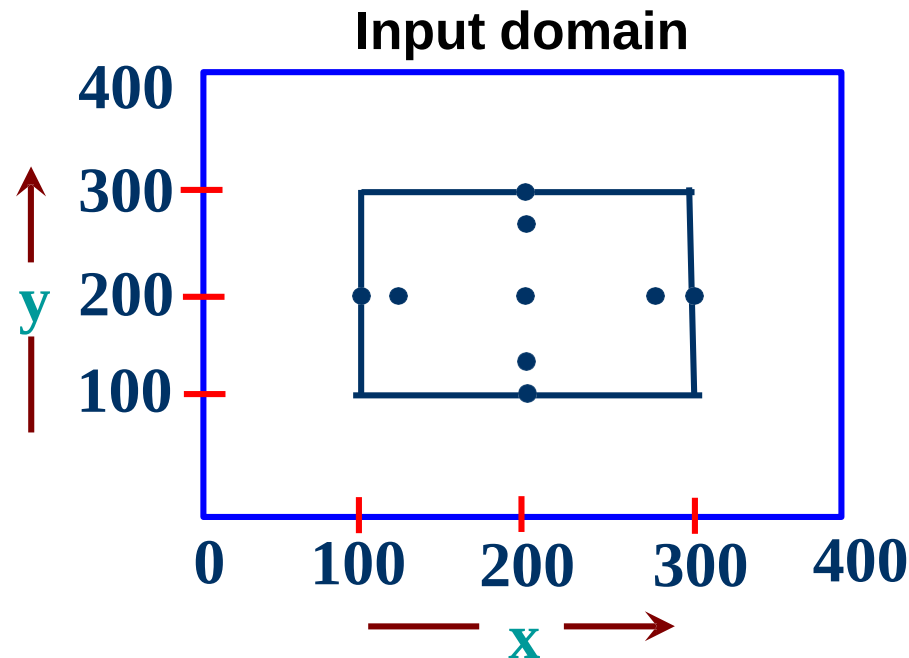


Fig. 5: Input domain of two variables x and y with boundaries [100,300] each

Software Testing

Example- 8.1

Consider a program for the determination of the nature of roots of a quadratic equation. Its input is a triple of positive integers (say a, b, c) and values may be from interval $[0, 100]$. The program output may have one of the following words.

[Not a quadratic equation; Real roots; Imaginary roots; Equal roots]

Design the boundary value test cases.

Software Testing

Solution

Quadratic equation will be of type:

$$ax^2+bx+c=0$$

Roots are real if $(b^2-4ac)>0$

Roots are imaginary if $(b^2-4ac)<0$

Roots are equal if $(b^2-4ac)=0$

Equation is not quadratic if $a=0$

Software Testing

The boundary value test cases are :

<i>Test Case</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>Expected output</i>
1	0	50	50	Not Quadratic
2	1	50	50	Real Roots
3	50	50	50	Imaginary Roots
4	99	50	50	Imaginary Roots
5	100	50	50	Imaginary Roots
6	50	0	50	Imaginary Roots
7	50	1	50	Imaginary Roots
8	50	99	50	Imaginary Roots
9	50	100	50	Equal Roots
10	50	50	0	Real Roots
11	50	50	1	Real Roots
12	50	50	99	Imaginary Roots
13	50	50	100	Imaginary Roots

Software Testing

Robustness testing

It is nothing but the extension of boundary value analysis. Here, we would like to see, what happens when the extreme values are exceeded with a value slightly greater than the maximum, and a value slightly less than minimum. It means, we want to go outside the legitimate boundary of input domain. This extended form of boundary value analysis is called robustness testing and shown in Fig. 6

There are four additional test cases which are outside the legitimate input domain. Hence total test cases in robustness testing are $6n+1$, where n is the number of input variables. So, 13 test cases are:

(200,99), (200,100), (200,101), (200,200), (200,299), (200,300)

(200,301), (99,200), (100,200), (101,200), (299,200), (300,200), (301,200)

Software Testing

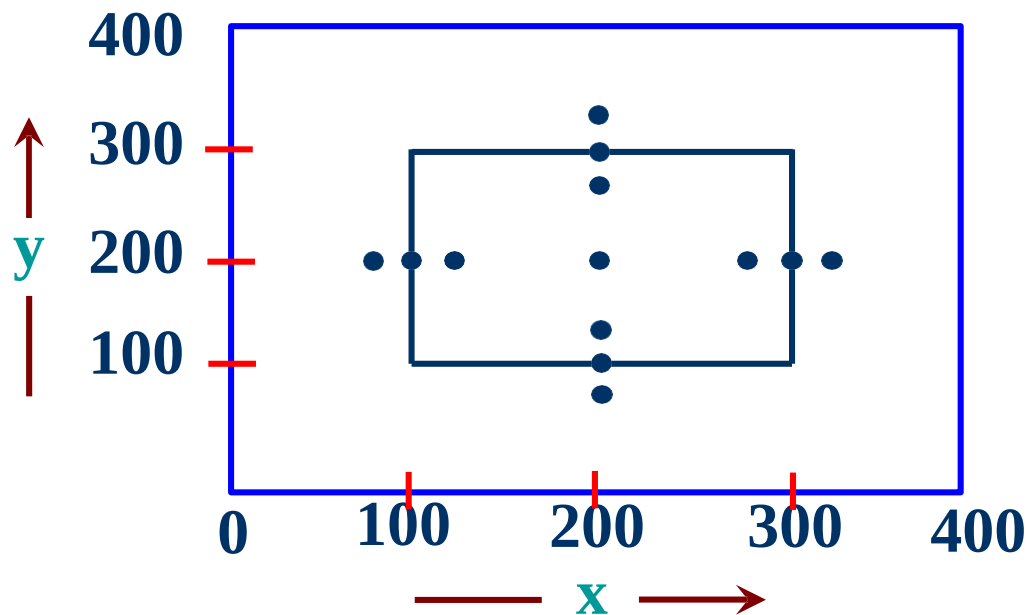


Fig. 8.6: Robustness test cases for two variables x and y with range $[100,300]$ each

Software Testing

Worst-case testing

If we reject “single fault” assumption theory of reliability and may like to see what happens when more than one variable has an extreme value. In electronic circuits analysis, this is called “worst case analysis”. It is more thorough in the sense that boundary value test cases are a proper subset of worst case test cases. It requires more effort. Worst case testing for a function of n variables generate 5^n test cases as opposed to $4n+1$ test cases for boundary value analysis. Our two variables example will have $5^2=25$ test cases and are given in table 1.

Software Testing

Table 1: Worst cases test inputs for two variables example

Test case number	Inputs		Test case number	Inputs	
	x	y		x	y
1	100	100	14	200	299
2	100	101	15	200	300
3	100	200	16	299	100
4	100	299	17	299	101
5	100	300	18	299	200
6	101	100	19	299	299
7	101	101	20	299	300
8	101	200	21	300	100
9	101	299	22	300	101
10	101	300	23	300	200
11	200	100	24	300	299
12	200	101	25	300	300
13	200	200	--		

Software Testing

Example - 8.4

Consider the program for the determination of nature of roots of a quadratic equation as explained in example 8.1. Design the Robust test case and worst test cases for this program.

Software Testing

Solution

Robust test cases are $6n+1$. Hence, in 3 variable input cases total number of test cases are 19 as given on next slide:

Software Testing

Test case	a	b	c	Expected Output
1	-1	50	50	Invalid input`
2	0	50	50	Not quadratic equation
3	1	50	50	Real roots
4	50	50	50	Imaginary roots
5	99	50	50	Imaginary roots
6	100	50	50	Imaginary roots
7	101	50	50	Invalid input
8	50	-1	50	Invalid input
9	50	0	50	Imaginary roots
10	50	1	50	Imaginary roots
11	50	99	50	Imaginary roots
12	50	100	50	Equal roots
13	50	101	50	Invalid input
14	50	50	-1	Invalid input
15	50	50	0	Real roots
16	50	50	1	Real roots
17	50	50	99	Imaginary roots
18	50	50	100	Imaginary roots
19	50	50	101	Invalid input

Software Testing

In case of worst test case total test cases are 5^n . Hence, 125 test cases will be generated in worst test cases. The worst test cases are given below:

Test Case	a	b	c	Expected output
1	0	0	0	Not Quadratic
2	0	0	1	Not Quadratic
3	0	0	50	Not Quadratic
4	0	0	99	Not Quadratic
5	0	0	100	Not Quadratic
6	0	1	0	Not Quadratic
7	0	1	1	Not Quadratic
8	0	1	50	Not Quadratic
9	0	1	99	Not Quadratic
10	0	1	100	Not Quadratic
11	0	50	0	Not Quadratic
12	0	50	1	Not Quadratic
13	0	50	50	Not Quadratic
14	0	50	99	Not Quadratic

(Contd.)...

Software Testing

Test Case	A	b	c	Expected output
15	0	50	100	Not Quadratic
16	0	99	0	Not Quadratic
17	0	99	1	Not Quadratic
18	0	99	50	Not Quadratic
19	0	99	99	Not Quadratic
20	0	99	100	Not Quadratic
21	0	100	0	Not Quadratic
22	0	100	1	Not Quadratic
23	0	100	50	Not Quadratic
24	0	100	99	Not Quadratic
25	0	100	100	Not Quadratic
26	1	0	0	Equal Roots
27	1	0	1	Imaginary
28	1	0	50	Imaginary
29	1	0	99	Imaginary
30	1	0	100	Imaginary
31	1	1	0	Real Roots

(Contd.)...

Software Testing

Test Case	A	b	C	Expected output
32	1	1	1	Imaginary
33	1	1	50	Imaginary
34	1	1	99	Imaginary
35	1	1	100	Imaginary
36	1	50	0	Real Roots
37	1	50	1	Real Roots
38	1	50	50	Real Roots
39	1	50	99	Real Roots
40	1	50	100	Real Roots
41	1	99	0	Real Roots
42	1	99	1	Real Roots
43	1	99	50	Real Roots
44`	1	99	99	Real Roots
45	1	99	100	Real Roots
46	1	100	0	Real Roots
47	1	100	1	Real Roots
48	1	100	50	Real Roots

(Contd.)...

Software Testing

Test Case	A	b	C	Expected output
49	1	100	99	Real Roots
50	1	100	100	Real Roots
51	50	0	0	Equal Roots
52	50	0	1	Imaginary
53	50	0	50	Imaginary
54	50	0	99	Imaginary
55	50	0	100	Imaginary
56	50	1	0	Real Roots
57	50	1	1	Imaginary
58	50	1	50	Imaginary
59	50	1	99	Imaginary
60	50	1	100	Imaginary
61	50	50	0	Real Roots
62	50	50	1	Real Roots
63	50	50	50	Imaginary
64	50	50	99	Imaginary
65	50	50	100	Imaginary

(Contd.)...

Software Testing

Test Case	A	b	C	Expected output
66	50	99	0	Real Roots
67	50	99	1	Real Roots
68	50	99	50	Imaginary
69	50	99	99	Imaginary
70	50	99	100	Imaginary
71	50	100	0	Real Roots
72	50	100	1	Real Roots
73	50	100	50	Equal Roots
74	50	100	99	Imaginary
75	50	100	100	Imaginary
76	99	0	0	Equal Roots
77	99	0	1	Imaginary
78	99	0	50	Imaginary
79	99	0	99	Imaginary
80	99	0	100	Imaginary
81	99	1	0	Real Roots
82	99	1	1	Imaginary

(Contd.)...

Software Testing

Test Case	A	b	C	Expected output
83	99	1	50	Imaginary
84	99	1	99	Imaginary
85	99	1	100	Imaginary
86	99	50	0	Real Roots
87	99	50	1	Real Roots
88	99	50	50	Imaginary
89	99	50	99	Imaginary
90	99	50	100	Imaginary
91	99	99	0	Real Roots
92	99	99	1	Real Roots
93	99	99	50	Imaginary Roots
94	99	99	99	Imaginary
95	99	99	100	Imaginary
96	99	100	0	Real Roots
97	99	100	1	Real Roots
98	99	100	50	Imaginary
99	99	100	99	Imaginary
100	99	100	100	Imaginary

(Contd.)...

Software Testing

Test Case	A	b	C	Expected output
101	100	0	0	Equal Roots
102	100	0	1	Imaginary
103	100	0	50	Imaginary
104	100	0	99	Imaginary
105	100	0	100	Imaginary
106	100	1	0	Real Roots
107	100	1	1	Imaginary
108	100	1	50	Imaginary
109	100	1	99	Imaginary
110	100	1	100	Imaginary
111	100	50	0	Real Roots
112	100	50	1	Real Roots
113	100	50	50	Imaginary
114	100	50	99	Imaginary
115	100	50	100	Imaginary
116	100	99	0	Real Roots
117	100	99	1	Real Roots
118	100	99	50	Imaginary

(Contd.)...

Software Testing

<i>Test Case</i>	<i>A</i>	<i>b</i>	<i>C</i>	<i>Expected output</i>
119	100	99	99	Imaginary
120	100	99	100	Imaginary
121	100	100	0	Real Roots
122	100	100	1	Real Roots
123	100	100	50	Imaginary
124	100	100	99	Imaginary
125	100	100	100	Imaginary

Software Testing

Equivalence Class Testing

In this method, input domain of a program is partitioned into a finite number of equivalence classes such that one can reasonably assume, but not be absolutely sure, that the test of a representative value of each class is equivalent to a test of any other value.

Two steps are required to implementing this method:

1. The equivalence classes are identified by taking each input condition and partitioning it into valid and invalid classes. For example, if an input condition specifies a range of values from 1 to 999, we identify one valid equivalence class $[1 < \text{item} < 999]$; and two invalid equivalence classes $[\text{item} < 1]$ and $[\text{item} > 999]$.
2. Generate the test cases using the equivalence classes identified in the previous step. This is performed by writing test cases covering all the valid equivalence classes. Then a test case is written for each invalid equivalence class so that no test contains more than one invalid class. This is to ensure that no two invalid classes mask each other.

Software Testing

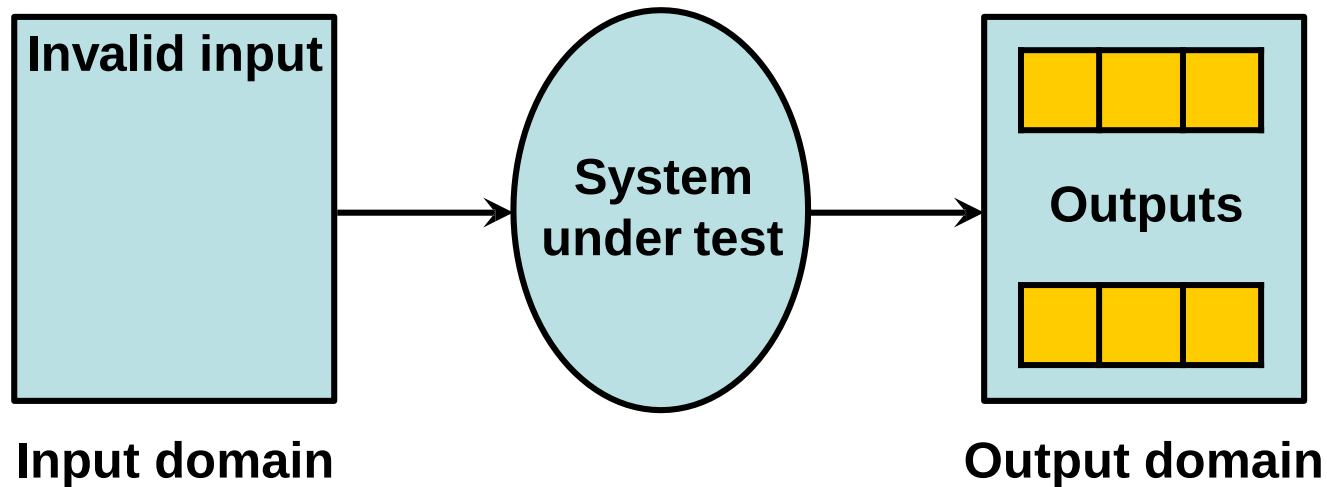


Fig. 7: Equivalence partitioning

Most of the time, equivalence class testing defines classes of the input domain. However, equivalence classes should also be defined for output domain. Hence, we should design equivalence classes based on input and output domain.

Software Testing

Example 8.7

Consider the program for the determination of nature of roots of a quadratic equation as explained in example 8.1. Identify the equivalence class test cases for output and input domains.

Software Testing

Solution

Output domain equivalence class test cases can be identified as follows:

$O_1 = \{ \langle a, b, c \rangle : \text{Not a quadratic equation if } a = 0 \}$

$O_1 = \{ \langle a, b, c \rangle : \text{Real roots if } (b^2 - 4ac) > 0 \}$

$O_1 = \{ \langle a, b, c \rangle : \text{Imaginary roots if } (b^2 - 4ac) < 0 \}$

$O_1 = \{ \langle a, b, c \rangle : \text{Equal roots if } (b^2 - 4ac) = 0 \}$

The number of test cases can be derived from above relations and shown below:

Test case	a	b	c	Expected output
1	0	50	50	Not a quadratic equation
2	1	50	50	Real roots
3	50	50	50	Imaginary roots
4	50	100	50	Equal roots

Software Testing

We may have another set of test cases based on input domain.

$$I_1 = \{a: a = 0\}$$

$$I_2 = \{a: a < 0\}$$

$$I_3 = \{a: 1 \leq a \leq 100\}$$

$$I_4 = \{a: a > 100\}$$

$$I_5 = \{b: 0 \leq b \leq 100\}$$

$$I_6 = \{b: b < 0\}$$

$$I_7 = \{b: b > 100\}$$

$$I_8 = \{c: 0 \leq c \leq 100\}$$

$$I_9 = \{c: c < 0\}$$

$$I_{10} = \{c: c > 100\}$$

Software Testing

Test Case	a	b	c	Expected output
1	0	50	50	Not a quadratic equation
2	-1	50	50	Invalid input
3	50	50	50	Imaginary Roots
4	101	50	50	invalid input
5	50	50	50	Imaginary Roots
6	50	-1	50	invalid input
7	50	101	50	invalid input
8	50	50	50	Imaginary Roots
9	50	50	-1	invalid input
10	50	50	101	invalid input

Here test cases 5 and 8 are redundant test cases. If we choose any value other than nominal, we may not have redundant test cases. Hence total test cases are $10+4=14$ for this problem.

Software Testing

Decision Table Based Testing

Condition Stub		Entry						
		True				False		
		True		False		True		False
		True	False	True	False	True	False	---
Action Stub	a ₁	X	X			X		
	a ₂	X		X			X	
	a ₃		X			X		
	a ₄				X		X	X

Table 2: Decision table terminology

Software Testing

Decision Table Based Testing

Decision table testing is a software testing technique used to test system behavior for different input combinations. This is a systematic approach where the different input combinations and their corresponding system behavior (Output) are captured in a tabular form. That is why it is also called as a **Cause-Effect** table where Cause and effects are captured for better test coverage.

Example:

Consider a 'Login' Page Functionality.

Here are the business rules.

1. On entering correct combination of ID & Password, user should be able to login successfully.
2. User is not allowed to login when any or both of the ID & Password are incorrect /blank. In such cases, it should show 'Invalid Credentials' message.

We created the following combinations of Conditions, Actions and the respective rules in the decision table.

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8	Rule 9
Conditions	TC01	TC02	TC03	TC04	TC05	TC06	TC07	TC08	TC09
UserID	Blank	Valid	Invalid	Blank	Blank	Invalid	Valid	Invalid	Valid
Password	Blank	Blank	Blank	Valid	Invalid	Valid	Invalid	Invalid	Valid
Actions									
Login Successfully									Execute
Error showing 'Invalid Credentials'	Execute	Execute	Execute	Execute	Execute	Execute	Execute	Execute	

In the above table, there are

2 conditions – UserID, Password

2 Actions – Login Successfully, Error showing 'Invalid Credentials' and

3 Options — Blank, Valid, Invalid.

So, the total number of test cases are as follows:

Options conditions i.e $3^2 = 9$ Test cases

You can collapse the above decision table based on certain rules as above.

So, let us try to reduce the number of test cases.

1.Rules 1,2, 3, 4, and 5 cover the same action Item “Invalid Credentials” with options Blank and Invalid. Hence, we can consider any one of these test cases TC01 OR TC02 OR TC03 OR TC04 OR TC05

2.Rules 6,7, and 8 cover the same action Item “Invalid Credentials” with options Valid and Invalid. Hence, we can consider any of these test cases TC06 OR TC07 OR TC08

3.Rule 9 covers the action item “Login Successfully” with all valid options. Hence, we should consider the test case TC09.

Finally, we will collapse the Decision Table as shown below:

	Rule 2	Rule 8	Rule9
Conditions	TC02	TC08	TC09
UserID	Valid	Invalid	Valid
Password	Blank	Invalid	Valid
Actions			
Login Successfully			Execute
Error showing 'Invalid Credentials'	Execute	Execute	

Software Testing

Test case design

C_1 : x,y,z are sides of a triangle?	N	Y							
C_2 : x = y?	--	Y				N			
C_3 : x = z?	--	Y		N		Y		N	
C_4 : y = z?	--	Y	N	Y	N	Y	N	Y	N
a_1 : Not a triangle	X								
a_2 : Scalene									X
a_3 : Isosceles					X		X	X	
a_4 : Equilateral		X							
a_5 : Impossible			X	X		X			

Table 3: Decision table for triangle problem

Software Testing

Conditions	F	T	T	T	T	T	T	T	T	T	T
$C_1: x < y + z?$											
$C_2: y < x + z?$	--	F	T	T	T	T	T	T	T	T	T
$C_3: z < x + y?$	--	--	F	T	T	T	T	T	T	T	T
$C_4: x = y?$	--	--	--	T	T	T	T	F	F	F	F
$C_5: x = z?$	--	--	--	T	T	F	F	T	T	F	F
$C_6: y = z?$	--	--	--	T	F	T	F	T	F	T	F
a_1: Not a triangle	X	X	X								
a_2: Scalene											X
a_3: Isosceles							X		X	X	
a_4: Equilateral				X							
a_5: Impossible					X	X		X			

Table 4: Modified decision table

Software Testing

Example 8.11

Consider a program for the determination of Previous date. Its input is a triple of day, month and year with the values in the range

$$1 \leq \text{month} \leq 12$$

$$1 \leq \text{day} \leq 31$$

$$1900 \leq \text{year} \leq 2025$$

The possible outputs are “Previous date” and “Invalid date”. Design the test cases using decision table based testing.

Software Testing

Solution

The input domain can be divided into following classes:

$I_1 = \{M_1: \text{month has 30 days}\}$

$I_2 = \{M_2: \text{month has 31 days except March, August and January}\}$

$I_3 = \{M_3: \text{month is March}\}$

$I_4 = \{M_4: \text{month is August}\}$

$I_5 = \{M_5: \text{month is January}\}$

$I_6 = \{M_6: \text{month is February}\}$

$I_7 = \{D_1: \text{day} = 1\}$

$I_8 = \{D_2: 2 \leq \text{day} \leq 28\}$

$I_9 = \{D_3: \text{day} = 29\}$

$I_{10} = \{D_4: \text{day} = 30\}$

$I_{11} = \{D_5: \text{day} = 31\}$

$I_{12} = \{Y_1: \text{year is a leap year}\}$

$I_{13} = \{Y_2: \text{year is a common year}\}$

Software Testing

The decision table is given below:

Sr.No.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
C ₁ : Months in	M ₁	M ₁	M ₁	M ₁	M ₁	M ₁	M ₁	M ₁	M ₁	M ₁	M ₂	M ₂	M ₂	M ₂	M ₂
C ₂ : days in	D ₁	D ₁	D ₂	D ₂	D ₃	D ₃	D ₄	D ₄	D ₅	D ₅	D ₁	D ₁	D ₂	D ₂	D ₃
C ₃ : year in	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁
a ₁ : Impossible									X	X					
a ₂ : Decrement day			X	X	X	X	X	X					X	X	X
a ₃ : Reset day to 31	X	X													
a ₄ : Reset day to 30											X	X			
a ₅ : Reset day to 29															
a ₆ : Reset day to 28															
a ₇ : decrement month	X	X									X	X			
a ₈ : Reset month to December															
a ₉ : Decrement year															

Software Testing

Sr.No.	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
C ₁ : Months in	M ₂	M ₂	M ₂	M ₂	M ₂	M ₃	M ₃	M ₃	M ₃	M ₃	M ₃	M ₃	M ₃	M ₃	M ₃
C ₂ : days in	D ₃	D ₄	D ₄	D ₅	D ₅	D ₁	D ₁	D ₂	D ₂	D ₃	D ₃	D ₄	D ₄	D ₅	D ₅
C ₃ : year in	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂
a ₁ : Impossible															
a ₂ : Decrement day	X	X	X	X	X			X	X	X	X	X	X	X	X
a ₃ : Reset day to 31															
a ₄ : Reset day to 30															
a ₅ : Reset day to 29						X									
a ₆ : Reset day to 28							X								
a ₇ : decrement month						X	X								
a ₈ : Reset month to December															
a ₉ : Decrement year															

Software Testing

Sr.No.	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45
C ₁ : Months in	M ₄	M ₄	M ₄	M ₄	M ₄	M ₄	M ₄	M ₄	M ₄	M ₄	M ₅	M ₅	M ₅	M ₅	M ₅
C ₂ : days in	D ₁	D ₁	D ₂	D ₂	D ₃	D ₃	D ₄	D ₄	D ₅	D ₅	D ₁	D ₁	D ₂	D ₂	D ₃
C ₃ : year in	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁
a ₁ : Impossible															
a ₂ : Decrement day			X	X	X	X	X	X	X	X			X	X	X
a ₃ : Reset day to 31	X	X									X	X			
a ₄ : Reset day to 30															
a ₅ : Reset day to 29															
a ₆ : Reset day to 28															
a ₇ : decrement month	X	X													
a ₈ : Reset month to December											X	X			
a ₉ : Decrement year											X	X			

Software Testing

Sr.No.	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60
C ₁ : Months in	M ₅	M ₅	M ₅	M ₅	M ₅	M ₆	M ₆	M ₆	M ₆	M ₆	M ₆	M ₆	M ₆	M ₆	M ₆
C ₂ : days in	D ₃	D ₄	D ₄	D ₅	D ₅	D ₁	D ₁	D ₂	D ₂	D ₃	D ₃	D ₄	D ₄	D ₅	D ₅
C ₃ : year in	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂	Y ₁	Y ₂
a ₁ : Impossible											X	X	X	X	X
a ₂ : Decrement day	X	X	X	X	X			X	X	X					
a ₃ : Reset day to 31						X	X								
a ₄ : Reset day to 30															
a ₅ : Reset day to 29															
a ₆ : Reset day to 28															
a ₇ : decrement month						X	X								
a ₈ : Reset month to December															
a ₉ : Decrement year															

Software Testing

Test case	Month	Day	Year	Expected output
1	June	1	1964	31 May, 1964
2	June	1	1962	31 May, 1962
3	June	15	1964	14 June, 1964
4	June	15	1962	14 June, 1962
5	June	29	1964	28 June, 1964
6	June	29	1962	28 June, 1962
7	June	30	1964	29 June, 1964
8	June	30	1962	29 June, 1962
9	June	31	1964	Impossible
10	June	31	1962	Impossible
11	May	1	1964	30 April, 1964
12	May	1	1962	30 April, 1962
13	May	15	1964	14 May, 1964
14	May	15	1962	14 May, 1962
15	May	29	1964	28 May, 1964

Software Testing

Test case	Month	Day	Year	Expected output
16	May	29	1962	28 May, 1962
17	May	30	1964	29 May, 1964
18	May	30	1962	29 May, 1962
19	May	31	1964	30 May, 1964
20	May	31	1962	30 May, 1962
21	March	1	1964	29 February, 1964
22	March	1	1962	28 February, 1962
23	March	15	1964	14 March, 1964
24	March	15	1962	14 March, 1962
25	March	29	1964	28 March, 1964
26	March	29	1962	28 March, 1962
27	March	30	1964	29 March, 1964
28	March	30	1962	29 March, 1962
29	March	31	1964	30 March, 1964
30	March	31	1962	30 March, 1962

Software Testing

Test case	Month	Day	Year	Expected output
31	August	1	1964	31 July, 1962
32	August	1	1962	31 July, 1964
33	August	15	1964	14 August, 1964
34	August	15	1962	14 August, 1962
35	August	29	1964	28 August, 1964
36	August	29	1962	28 August, 1962
37	August	30	1964	29 August, 1964
38	August	30	1962	29 August, 1962
39	August	31	1964	30 August, 1964
40	August	31	1962	30 August, 1962
41	January	1	1964	31 December, 1964
42	January	1	1962	31 December, 1962
43	January	15	1964	14 January, 1964
44	January	15	1962	14 January, 1962
45	January	29	1964	28 January, 1964

Software Testing

Test case	Month	Day	Year	Expected output
46	January	29	1962	28 January, 1962
47	January	30	1964	29 January, 1964
48	January	30	1962	29 January, 1962
49	January	31	1964	30 January, 1964
50	January	31	1962	30 January, 1962
51	February	1	1964	31 January, 1964
52	February	1	1962	31 January, 1962
53	February	15	1964	14 February, 1964
54	February	15	1962	14 February, 1962
55	February	29	1964	28 February, 1964
56	February	29	1962	Impossible
57	February	30	1964	Impossible
58	February	30	1962	Impossible
59	February	31	1964	Impossible
60	February	31	1962	Impossible

Software Testing

Static Testing, a software testing technique in which the software is tested without executing the code. (OR)

It checks the syntax of the code, coding standards, code optimization, etc.

Informal Review:

- Doesn't follow any process to find errors in the document, you just review the document and give informal comments on it.
- Applied many times during the early stages of the life cycle of the document.
- A two person team can conduct an informal review and in later stages more people are involved.
- The goal is to keep the author and to improve the quality of the document.

Software Testing

Formal Review

Formal reviews follow a formal process. It is well structured and regulated (Controlled). A formal review process consists of six main steps:

1. Planning
2. Kick-off
3. Preparation
4. Review meeting
5. Rework
6. Follow-up

Software Testing

Technical Review

- A team consisting of your peers, review the technical specification of the software product and checks whether it is suitable for the project.
- They try to find any difference in the specifications and standards followed.
- This review concentrates mainly on the technical document related to the software such as Test Strategy, Test Plan and requirement specification documents.

Software Testing

Code Walkthrough

- Code walk through is an informal code analysis technique. In this, after a module has been coded, it is successfully compiled and all syntax errors are eliminated.
- The main objective of the walk through is to discover the algorithmic and logical errors in the code.
- The author of the work product explains the product to his team.
- Participants can ask questions if any.
- Meeting is led by the author.
- Scribe makes note of review comments.

Software Testing

Code Inspection

- The aim of code inspection is to discover some common types of errors caused due to oversight and improper programming.
- Meeting is led by trained moderator.
- This review is a formal type of review where it follows strict process to find the defects.
- Reviewers have checklist to review the work products .
- They record the defect and inform the participants to rectify those errors.

List of some commonly programming errors which can be checked during code inspection:

- Use of uninitialized variables.
- Jumps into loops.
- Non terminating loops.
- Array indices out of bounds.
- Mismatches b/w actual and formal parameters in procedure calls.

CODE INSPECTION CHECKLIST

Project Name:

Program Unit Name:

Program Author's Name:

Yes

No

Comments

DATA HANDLING

Correct initialization of:

variables,

arrays,

pointers,

counters,

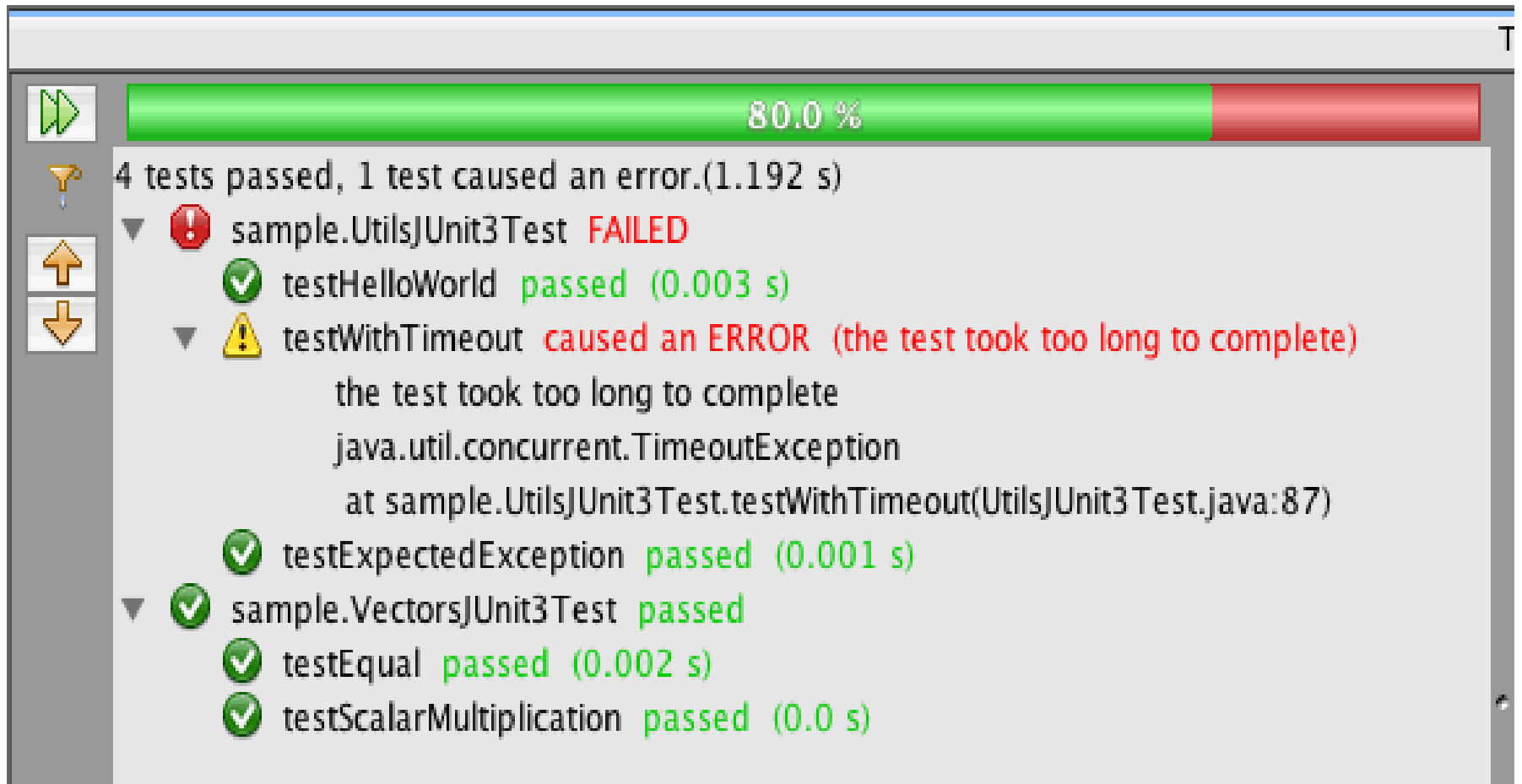
flags,

strings.

Arrays			
Subscripts are within bounds.			
Subscripts are in the correct order.			
Pointers			
Pointer references are valid.			
Strings			
Strings are free of off-by-one errors.			
Indexing is within bounds.			

CONTROL FLOW			
Subroutines			
The correct subroutines are called.			
Subroutines are in the correct order.			
Gotos			
Gotos go to the correct location.			
Gotos go backward, not forward.			
Loops			
The number of loop iterations is correct.			
Every loop will terminate.			
Criteria for ending each loop are correct.			
Every loop can be executed.			

Jtest – Static Testing Tool



The image shows the Jtest static testing tool interface. At the top, a green progress bar indicates 80.0% completion. Below the progress bar, a summary line states "4 tests passed, 1 test caused an error.(1.192 s)". The test results are listed in a tree view on the left, with icons for expand/collapse, filter, and scroll. The tree shows two test classes: `sample.UtilsJUnit3Test` and `sample.VectorsJUnit3Test`. `sample.UtilsJUnit3Test` has two tests: `testHelloWorld` (passed) and `testWithTimeout` (failed). `testWithTimeout` failed with a `java.util.concurrent.TimeoutException` at `sample.UtilsJUnit3Test.testWithTimeout(UtilsJUnit3Test.java:87)`. `sample.VectorsJUnit3Test` has three tests: `testExpectedException` (passed), `testEqual` (passed), and `testScalarMultiplication` (passed).

80.0 %

4 tests passed, 1 test caused an error.(1.192 s)

- ▼  `sample.UtilsJUnit3Test` **FAILED**
 - ✓ `testHelloWorld` passed (0.003 s)
 - ▼  `testWithTimeout` **caused an ERROR** (the test took too long to complete)
 - the test took too long to complete
 - `java.util.concurrent.TimeoutException`
 - at `sample.UtilsJUnit3Test.testWithTimeout(UtilsJUnit3Test.java:87)`
 - ✓ `testExpectedException` passed (0.001 s)
- ▼  `sample.VectorsJUnit3Test` **passed**
 - ✓ `testEqual` passed (0.002 s)
 - ✓ `testScalarMultiplication` passed (0.0 s)

Software Testing

Structural Testing

A complementary approach to functional testing is called structural / white box testing. It permits us to examine the internal structure of the program.

Path Testing

Path testing is the name given to a group of test techniques based on judiciously selecting a set of test paths through the program. If the set of paths is properly chosen, then it means that we have achieved some measure of test thoroughness.

This type of testing involves:

1. generating a set of paths that will cover every branch in the program.
2. finding a set of test cases that will execute every path in the set of program paths.

Software Testing

Control Flow Graph

The control flow of a program can be analysed using a graphical representation known as flow graph. The flow graph is a directed graph in which nodes are either entire statements or fragments of a statement, and edges represents flow of control.

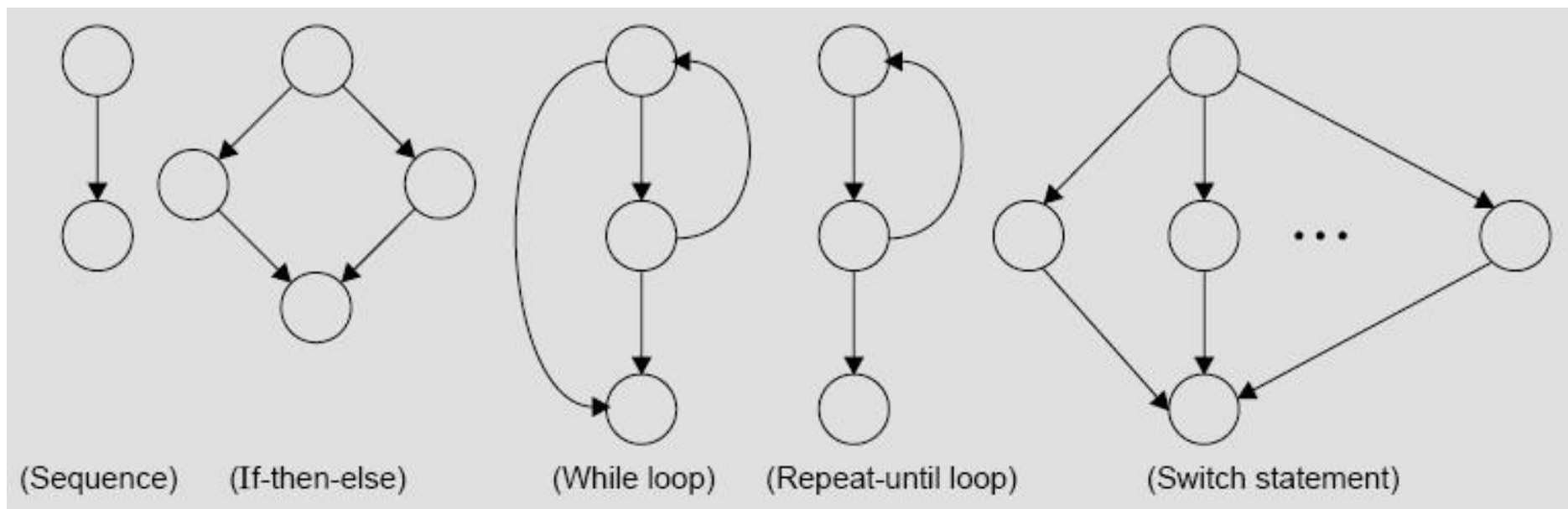


Fig. 14: The basic construct of the flow graph

Software Testing

Example 8.13

Consider the problem for the determination of the nature of roots of a quadratic equation. Its input a triple of positive integers (say a, b, c) and value may be from interval $[0, 100]$.

The program is given in fig. 19. The output may have one of the following words:

[Not a quadratic equation; real roots; Imaginary roots; Equal roots]

Draw the flow graph and DD path graph. Also find independent paths from the DD Path graph.

Software Testing

```
#include <conio.h>
#include <math.h>
1   int main()
2   {
3       int a,b,c,validInput=0,d;
4       double D;
5       printf("Enter the 'a' value: ");
6       scanf("%d",&a);
7       printf("Enter the 'b' value: ");
8       scanf("%d",&b);
9       printf("Enter the 'c' value: ");
10      scanf("%d",&c);
11      if ((a >= 0) && (a <= 100) && (b >= 0) && (b <= 100) && (c >= 0)
        && (c <= 100)) {
12          validInput = 1;
13          if (a == 0) {
14              validInput = -1;
15          }
16      }
17      if (validInput==1) {
18          d = b*b - 4*a*c;
19          if (d == 0) {
20              printf("The roots are equal and are r1 = r2 = %f\n",
                  -b/(2*(float) a));
```

Cont...
87

Software Testing

```
21     }
22     else if ( d > 0 ) {
23         D=sqrt(d);
24         printf("The roots are real and are r1 = %f and r2 = %f\n",
                (-b-D)/(2* a), (-b+D)/(2* a));
25     }
26     else {
27         D=sqrt(-d)/(2*a);
28         printf("The roots are imaginary and are r1 = (%f,%f) and
                r2 = (%f,%f)\n", -b/(2.0*a),D,-b/(2.0*a),-D);
29     }
30 }
31 else if (validInput == -1) {
32     printf("The vlaues do not constitute a Quadratic equation.");
33 }
34 else {
35     printf("The inputs belong to invalid range.");
36 }
37 getch();
38 return 1;
39 }
```

Fig. 19: Code of quadratic equation problem

Software Testing

Solution

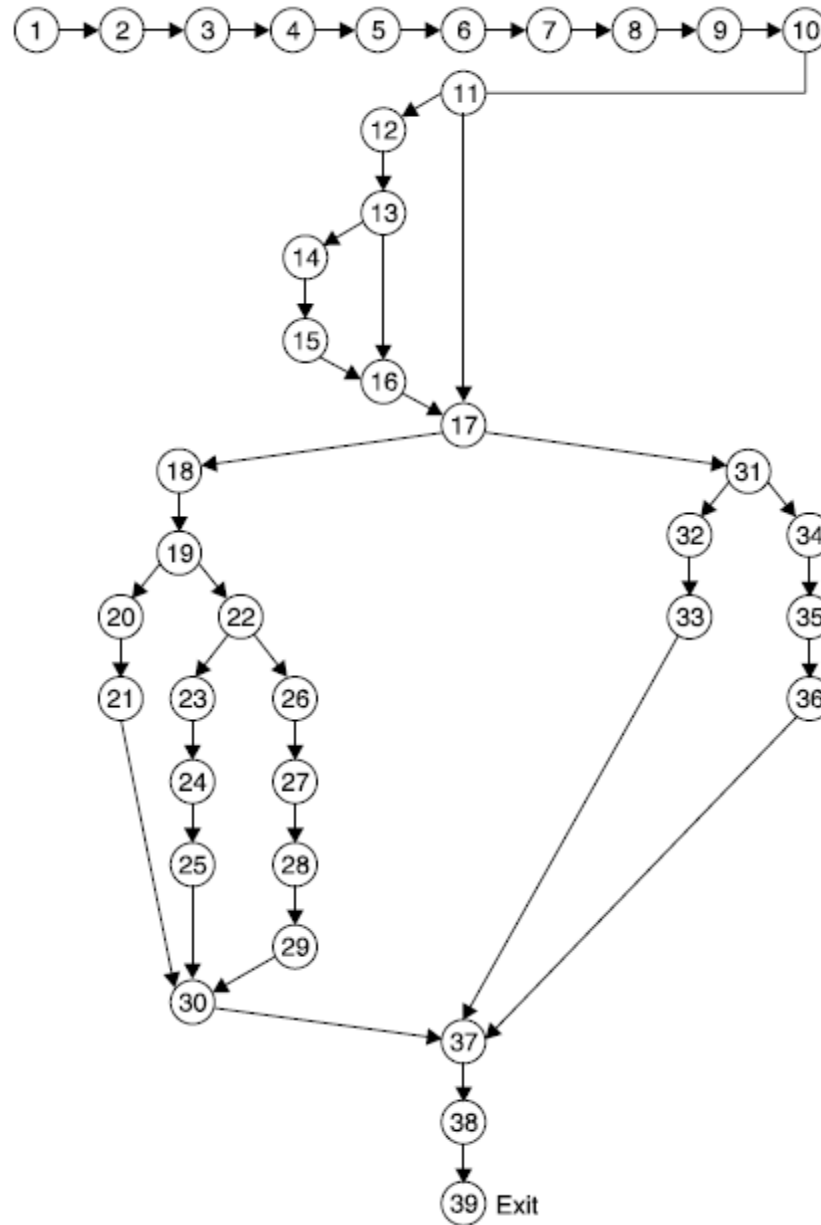


Fig. 19 (a) : Program flow graph

Software Testing

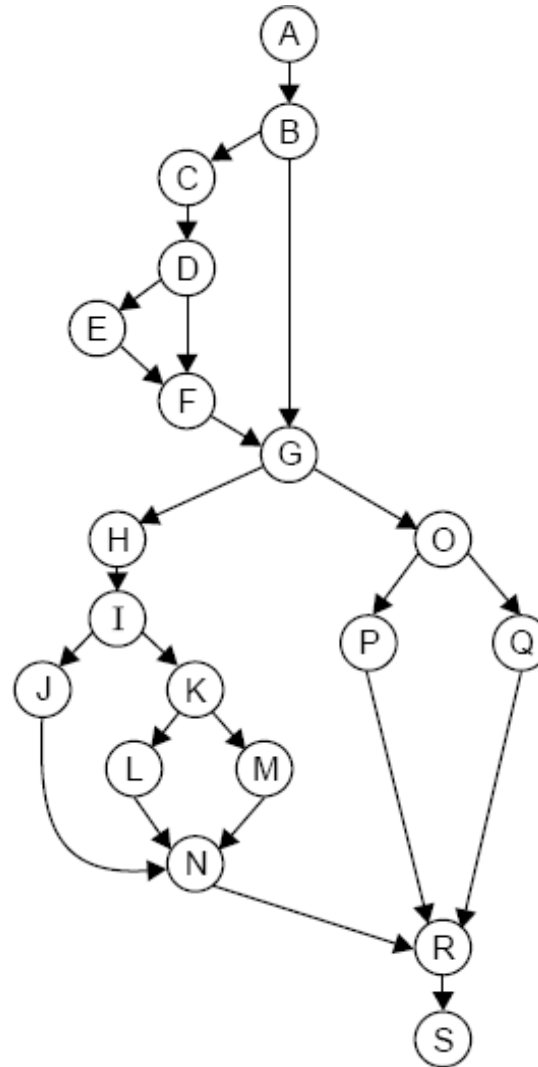


Fig. 19 (b) : DD Path graph

Software Testing

The mapping table for DD path graph is:

Flow graph nodes	DD Path graph corresponding node	Remarks
1 to 10	A	Sequential nodes
11	B	Decision node
12	C	Intermediate node
13	D	Decision node
14,15	E	Sequential node
16	F	Two edges are combined here
17	G	Two edges are combined and decision node
18	H	Intermediate node
19	I	Decision node
20,21	J	Sequential node
22	K	Decision node
23,24,25	L	Sequential node

Cont....

Software Testing

Flow graph nodes	DD Path graph corresponding node	Remarks
26,27,28,29	M	Sequential nodes
30	N	Three edges are combined
31	O	Decision node
32,33	P	Sequential node
34,35,36	Q	Sequential node
37	R	Three edges are combined here
38,39	S	Sequential nodes with exit node

Independent paths are:

(i) ABGOQRS

(iii) ABCDFGOQRS

(v) ABGHIJNRS

(vi) ABGHIKMNRS

(ii) ABGOPRS

(iv) ABCDEFGOPRS

(vi) ABGHIKLNRS

Software Testing

Cyclomatic Complexity

McCabe's cyclomatic metric $V(G) = e - n + 2P$.

For example, a flow graph shown in in Fig. 21 with entry node 'a' and exit node 'f'.

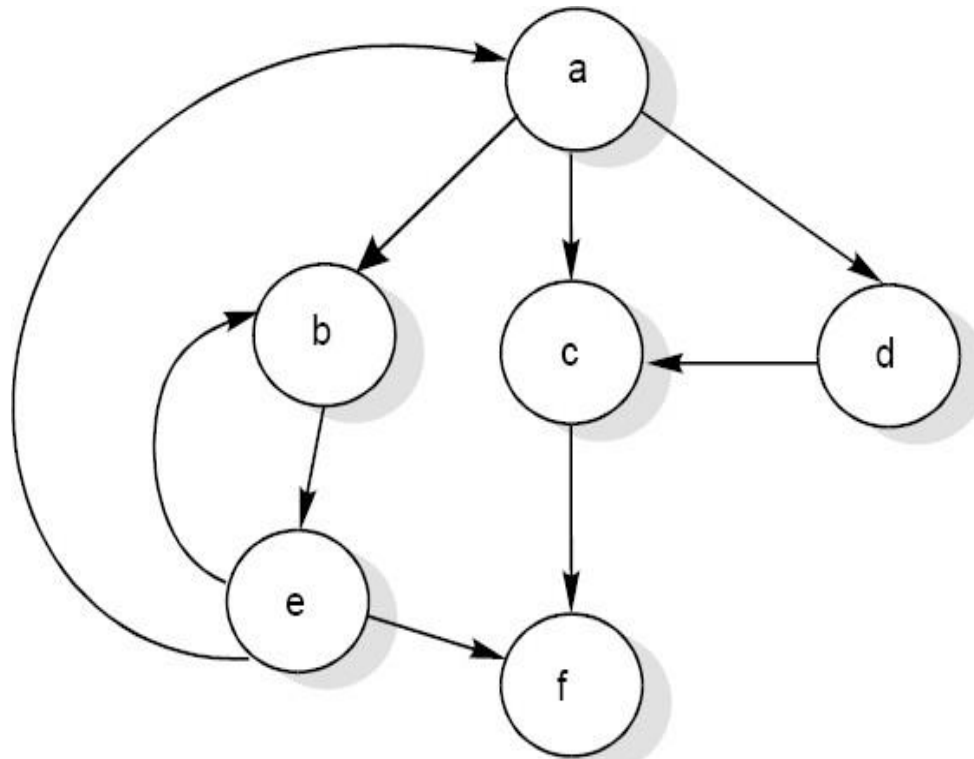


Fig. 21: Flow graph

Software Testing

The value of cyclomatic complexity can be calculated as :

$$V(G) = 9 - 6 + 2 = 5$$

Here $e = 9$, $n = 6$ and $P = 1$

There will be five independent paths for the flow graph illustrated in Fig. 21.

- Path 1 :** $a c f$
- Path 2 :** $a b e f$
- Path 3 :** $a d c f$
- Path 4 :** $a b e a c f$ or $a b e a b e f$
- Path 5 :** $a b e b e f$

Software Testing

Several properties of cyclomatic complexity are stated below:

1. $V(G) \geq 1$
2. $V(G)$ is the maximum number of independent paths in graph G .
3. Inserting & deleting functional statements to G does not affect $V(G)$.
4. G has only one path if and only if $V(G)=1$.
5. Inserting a new row in G increases $V(G)$ by unity.
6. $V(G)$ depends only on the decision structure of G .

Software Testing

The role of P in the complexity calculation $V(G)=e-n+2P$ is required to be understood correctly. We define a flow graph with unique entry and exit nodes, all nodes reachable from the entry, and exit reachable from all nodes. This definition would result in all flow graphs having only one connected component. One could, however, imagine a main program M and two called subroutines A and B having a flow graph shown in Fig. 22.

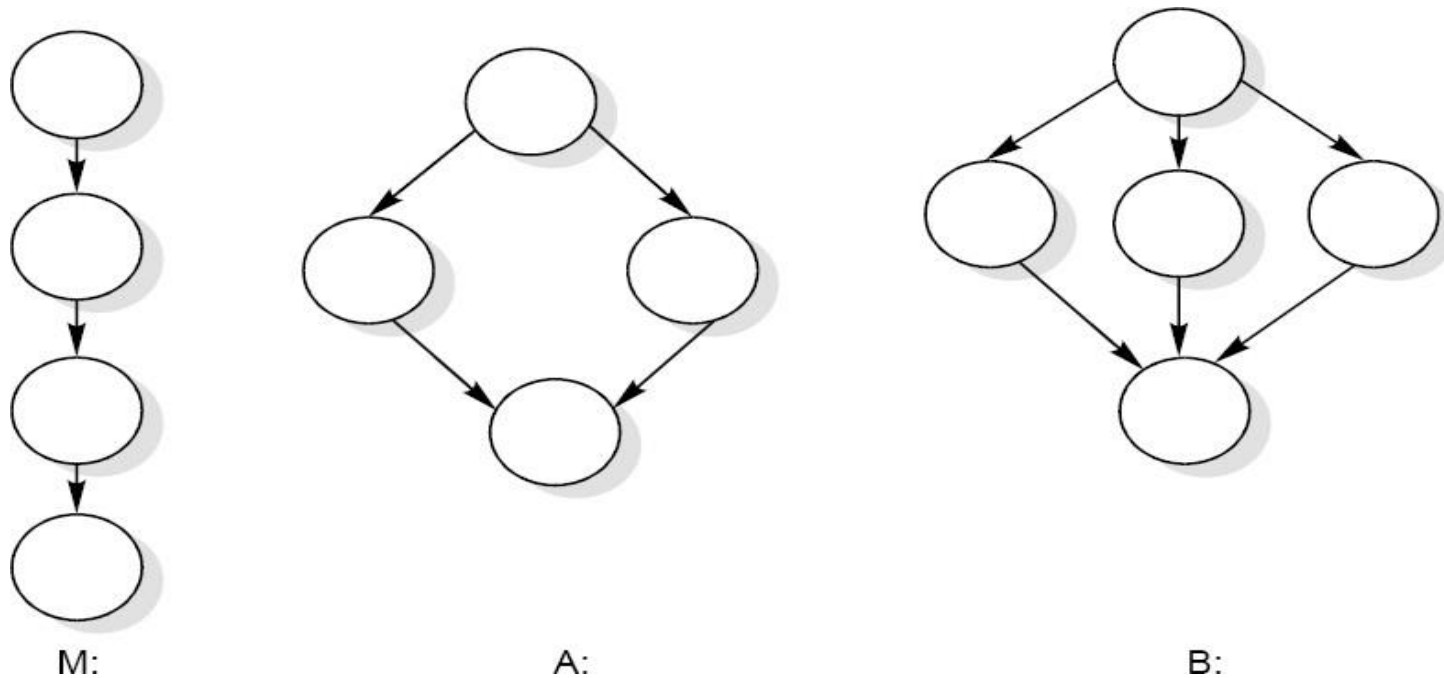


Fig. 22

Software Testing

Let us denote the total graph above with 3 connected components as

$$\begin{aligned} V(M \cup A \cup B) &= e - n + 2P \\ &= 13 - 13 + 2 \cdot 3 \\ &= 6 \end{aligned}$$

This method with $P \neq 1$ can be used to calculate the complexity of a collection of programs, particularly a hierarchical nest of subroutines.

Software Testing

Notice that $V(M \cup A \cup B) = V(M) + V(A) + V(B) = 6$. In general, the complexity of a collection C of flow graphs with K connected components is equal to the summation of their complexities. To see this let $C_i, 1 \leq i \leq K$ denote the k distinct connected component, and let e_i and n_i be the number of edges and nodes in the i th-connected component. Then

$$\begin{aligned} V(C) &= e - n + 2p = \sum_{i=1}^k e_i - \sum_{i=1}^k n_i + 2K \\ &= \sum_{i=1}^k (e_i - n_i + 2) = \sum_{i=1}^k V(C_i) \end{aligned}$$

Software Testing

Two alternate methods are available for the complexity calculations.

1. Cyclomatic complexity $V(G)$ of a flow graph G is equal to the number of predicate (decision) nodes plus one.

$$V(G) = \Pi + 1$$

Where Π is the number of predicate nodes contained in the flow graph G .

2. Cyclomatic complexity is equal to the number of regions of the flow graph.

Software Testing

Example 8.15

Consider a flow graph given in Fig. 23 and calculate the cyclomatic complexity by all three methods.

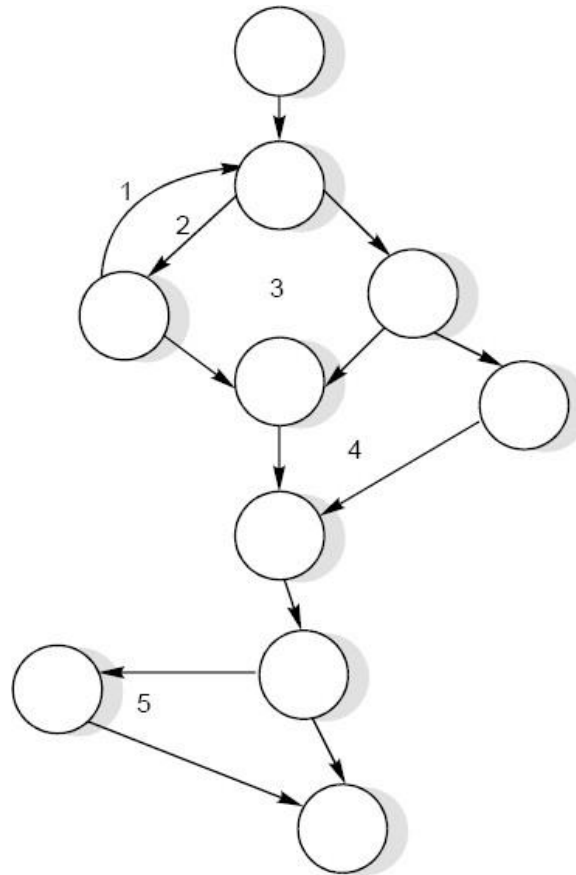


Fig. 23

Software Testing

Solution

Cyclomatic complexity can be calculated by any of the three methods.

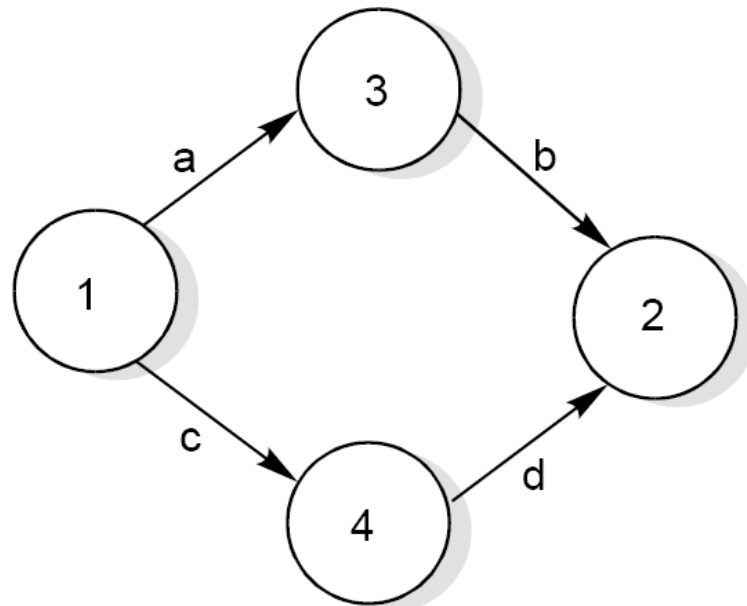
1. $V(G) = e - n + 2P$
 $= 13 - 10 + 2 = 5$
2. $V(G) = \pi + 1$
 $= 4 + 1 = 5$
3. $V(G) = \text{number of regions}$
 $= 5$

Therefore, complexity value of a flow graph in Fig. 23 is 5.

Software Testing

Graph Matrices

A graph matrix is a square matrix with one row and one column for every node in the graph. The size of the matrix (i.e., the number of rows and columns) is equal to the number of nodes in the flow graph. Some examples of graphs and associated matrices are shown in fig. 24.



Flow graph

(a)

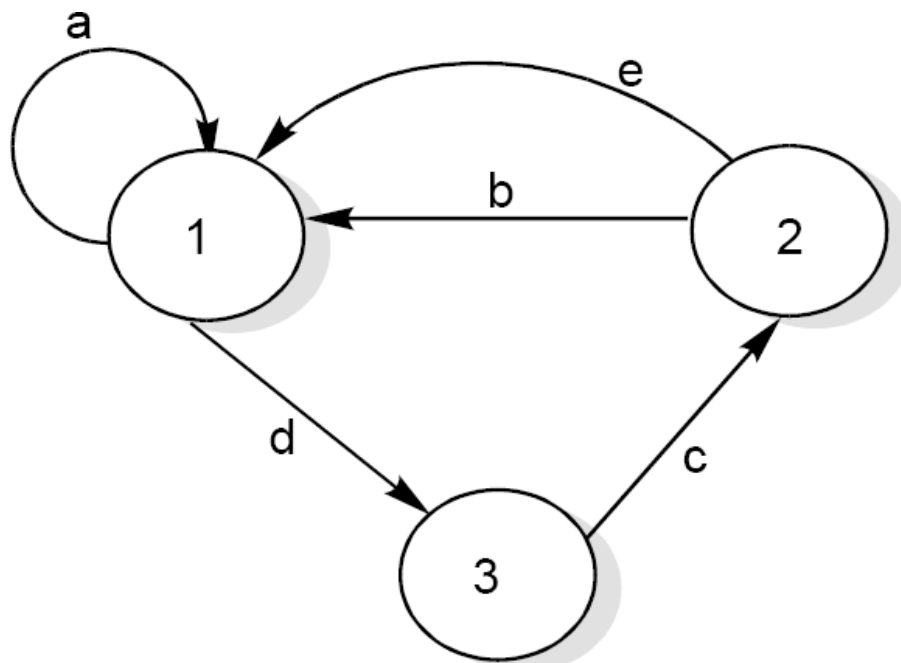
	1	2	3	4
1			a	c
2				
3		b		
4		d		

Graph Matrix

Fig. 24 (a): Flow graph and graph matrices

(Contd.)...

Software Testing



Flow graph

	1	2	3
1	a		d
2	b + e		
3		c	

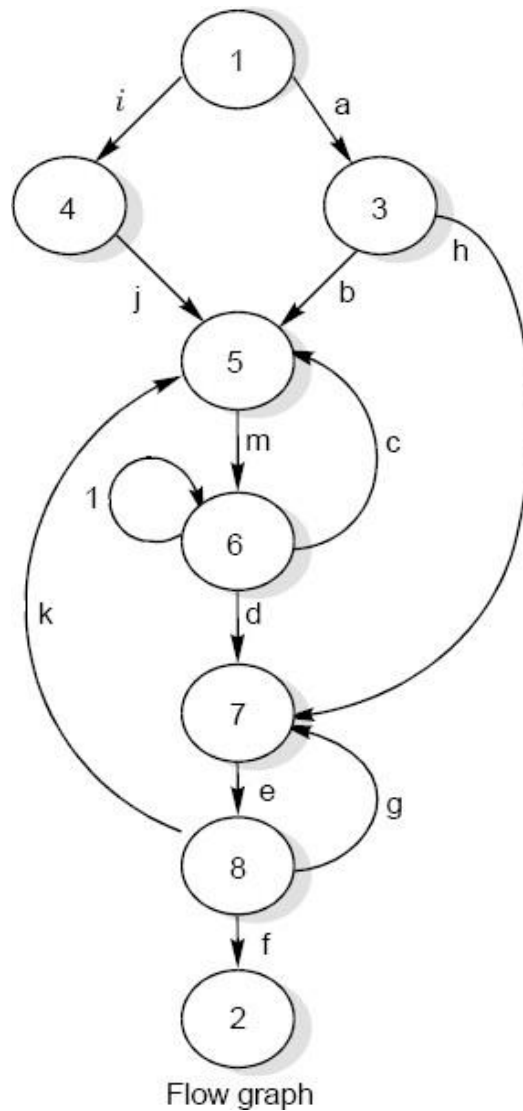
Graph Matrix

(b)

Fig. 24 (b): Flow graph and graph matrices

(Contd.)...

Software Testing



	1	2	3	4	5	6	7	8
1			a	i				
2								
3					b		h	
4					j			
5						m		
6					c	l	d	
7								e
8		f			k		g	

Graph Matrix

(c)

Fig. 24 (c): Flow graph and graph matrices

Software Testing

Connections

	1	2	3	4	5	6	7	8	
1			1	1					$2 - 1 = 1$
2									
3					1		1		$2 - 1 = 1$
4					1				$1 - 1 = 0$
5						1			$1 - 1 = 0$
6					1	1	1		$3 - 1 = 2$
7								1	$1 - 1 = 0$
8		1			1		1		$3 - 1 = 2$
									$6 + 1 = 7$

Fig. 25 : Connection matrix of flow graph shown in Fig. 24 (c)

Software Testing

	1	2	3	4
1			a	c
2				
3		b		
4		d		

[A]

	1	2	3	4
1		ab + cd		
2				
3				
4				

$[A]^2$

The square matrix represent that there are two path ab and cd from node 1 to node 2.

Software Testing

Data Flow Testing

Data flow testing is another form of structural testing. It has nothing to do with data flow diagrams.

- i. Statements where variables receive values.
- ii. Statements where these values are used or referenced.

As we know, variables are defined and referenced throughout the program. We may have few define/ reference anomalies:

- i. A variable is defined but not used/ referenced.
- ii. A variable is used but never defined.
- iii. A variable is defined twice before it is used.

Software Testing

Definitions

The definitions refer to a program P that has a program graph $G(P)$ and a set of program variables V . The $G(P)$ has a single entry node and a single exit node. The set of all paths in P is $PATHS(P)$

- (i) **Defining Node:** Node $n \in G(P)$ is a defining node of the variable $v \in V$, written as $DEF(v, n)$, if the value of the variable v is defined at the statement fragment corresponding to node n .
- (ii) **Usage Node:** Node $n \in G(P)$ is a usage node of the variable $v \in V$, written as $USE(v, n)$, if the value of the variable v is used at statement fragment corresponding to node n . A usage node $USE(v, n)$ is a predicate use (denote as p) if statement n is a predicate statement otherwise $USE(v, n)$ is a computation use (denoted as c).

Software Testing

- (iii) **Definition use:** A definition use path with respect to a variable v (denoted du -path) is a path in $PATHS(P)$ such that, for some $v \in V$, there are define and usage nodes $DEF(v, m)$ and $USE(v, n)$ such that m and n are initial and final nodes of the path.
- (iv) **Definition clear :** A definition clear path with respect to a variable v (denoted dc -path) is a definition use path in $PATHS(P)$ with initial and final nodes $DEF(v, m)$ and $USE(v, n)$, such that no other node in the path is a defining node of v .

The du -paths and dc -paths describe the flow of data across source statements from points at which the values are defined to points at which the values are used. The du -paths that are not definition clear are potential trouble spots.

Software Testing

Hence, our objective is to find all du-paths and then identify those du-paths which are not dc-paths. The steps are given in Fig. 27. We may like to generate specific test cases for du-paths that are not dc-paths.

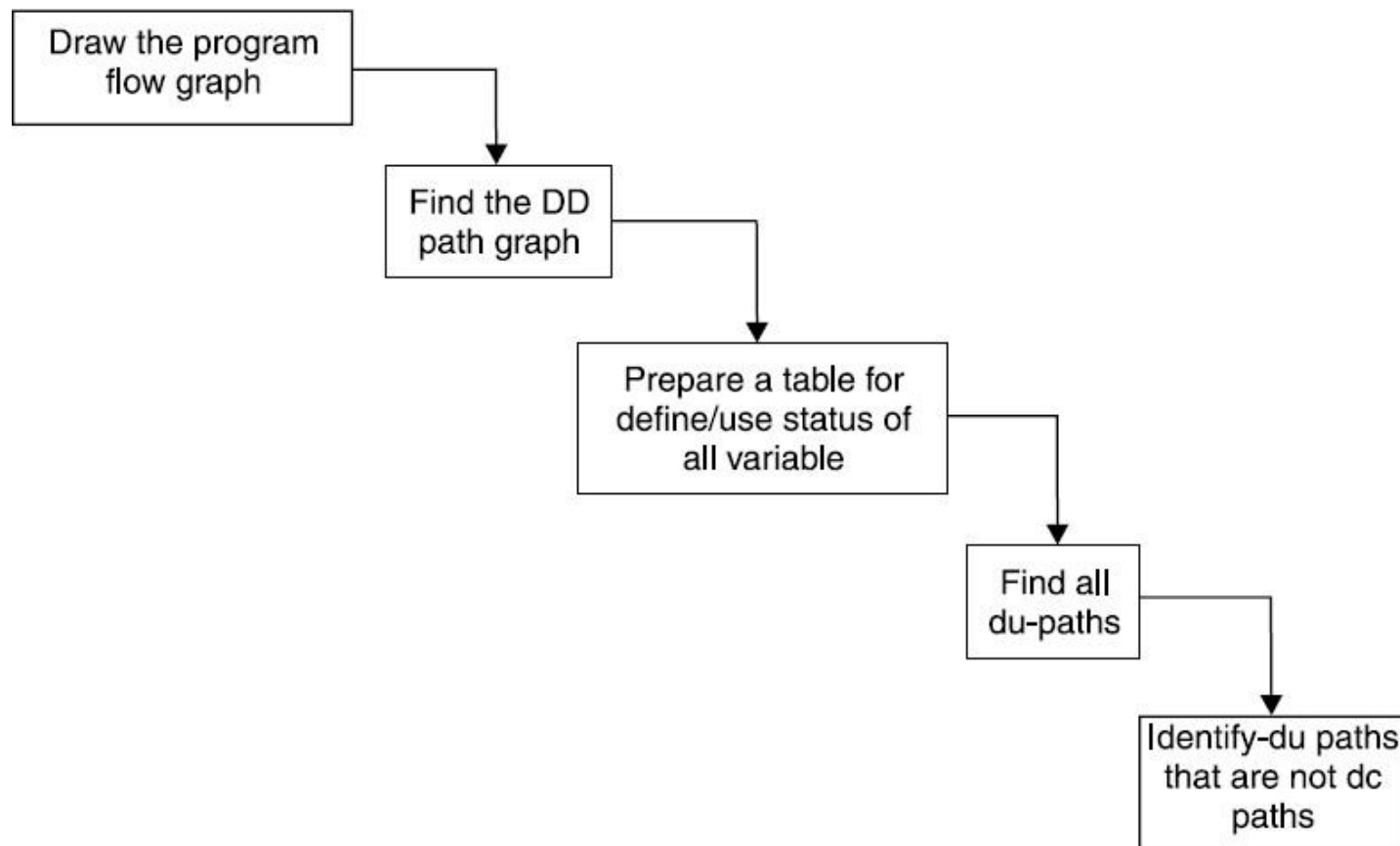


Fig. 27 : Steps for data flow testing

Software Testing

Example 8.20

Consider the program of the determination of the nature of roots of a quadratic equation. Its input is a triple of positive integers (say a, b, c) and values for each of these may be from interval $[0, 100]$. The program is given in Fig. 19. The output may have one of the option given below:

- (i) Not a quadratic program
- (ii) real roots
- (iii) imaginary roots
- (iv) equal roots
- (v) invalid inputs

Find all du-paths and identify those du-paths that are definition clear.

Software Testing

Solution

Step I: The program flow graph is given in Fig. 19 (a). The variables used in the program are a,b,c,d, validinput, D.

Step II: DD Path graph is given in Fig. 19(b). The cyclomatic complexity of this graph is 7 indicating there are seven independent paths.

Step III: Define/use nodes for all variables are given below:

Variable	Defined at node	Used at node
a	6	11,13,18,20,24,27,28
b	8	11,18,20,24,28
c	10	11,18
d	18	19,22,23,27
D	23, 27	24,28
Validinput	3, 12, 14	17,31

Software Testing

Step IV: The du-paths are identified and are named by their beginning and ending nodes using Fig. 19 (a).

Variable	Path (beginning, end) nodes	Definition clear ?
a	6, 11	Yes
	6, 13	Yes
	6, 18	Yes
	6, 20	Yes
	6, 24	Yes
	6, 27	Yes
	6, 28	Yes
b	8, 11	Yes
	8, 18	Yes
	8, 20	Yes
	8, 24	Yes
	8, 28	Yes

(Contd.)...

Software Testing

Variable	Path (beginning, end) nodes	Definition clear ?
c	10, 11	Yes
	10, 18	Yes
d	18, 19	Yes
	18, 22	Yes
	18, 23	Yes
	18, 27	Yes
D	23, 24	Yes
	23, 28	Path not possible
	27, 24	Path not possible
	27, 28	Yes
validinput	3, 17	no
	3, 31	no
	12, 17	no
	12, 31	no
	14, 17	yes
	14, 31	yes

Software Testing

Mutation Testing

Mutation testing is a fault based technique that is similar to fault seeding, except that mutations to program statements are made in order to determine properties about test cases. it is basically a fault simulation technique.

Multiple copies of a program are made, and each copy is altered; this altered copy is called a mutant. Mutants are executed with test data to determine whether the test data are capable of detecting the change between the original program and the mutated program.

A mutant that is detected by a test case is termed “killed” and the goal of mutation procedure is to find a set of test cases that are able to kill groups of mutant programs.

Software Testing

When we mutate code there needs to be a way of measuring the degree to which the code has been modified. For example, if the original expression is $x+1$ and the mutant for that expression is $x+2$, that is a lesser change to the original code than a mutant such as $(c*22)$, where both the operand and the operator are changed. We may have a ranking scheme, where a first order mutant is a single change to an expression, a second order mutant is a mutation to a first order mutant, and so on. High order mutants becomes intractable and thus in practice only low order mutants are used.

One difficulty associated with whether mutants will be killed is the problem of reaching the location; if a mutant is not executed, it cannot be killed. Special test cases are to be designed to reach a mutant. For example, suppose, we have the code.

Read (a,b,c);

If($a>b$) and ($b=c$) then

$x:=a*b*c$; (make mutants; $m_1, m_2, m_3 \dots\dots$)

Software Testing

To execute this, input domain must contain a value such that a is greater than b and b equals c . If input domain does not contain such a value, then all mutants made at this location should be considered equivalent to the original program, because the statement $x:=a*b*c$ is dead code (code that cannot be reached during execution). If we make the mutant $x+y$ for $x+1$, then we should take care about the value of y which should not be equal to 1 for designing a test case.

The manner by which a test suite is evaluated (scored) via mutation testing is as follows: for a specified test suite and a specific set of mutants, there will be three types of mutants in the code i.e., killed or dead, live, equivalent. The sum of the number of live, killed, and equivalent mutants will be the total number of mutants created. The score associated with a test suite T and mutants M is simply.

$$\frac{\#killed}{\#total - \#equivalent} \times 100\%$$

Software Testing

The Art of Debugging

The goal of testing is to identify errors (bugs) in the program. The process of testing generates symptoms, and a program's failure is a clear symptom of the presence of an error. After getting a symptom, we begin to investigate the cause and place of that error. After identification of place, we examine that portion to identify the cause of the problem. This process is called debugging.

Debugging Techniques

Pressman explained few characteristics of bugs that provide some clues.

1. "The symptom and the cause may be geographically remote. That is, the symptom may appear in one part of a program, while the cause may actually be located in other part. Highly coupled program structures may complicate this situation.
2. The symptom may disappear (temporarily) when another error is corrected.

Software Testing

3. The symptom may actually be caused by non errors (e.g. round off inaccuracies).
4. The symptom may be caused by a human error that is not easily traced.
5. The symptom may be a result of timing problems rather than processing problems.
6. It may be difficult to accurately reproduce input conditions (e.g. a real time application in which input ordering is indeterminate).
7. The symptom may be intermittent. This is particularly common in embedded system that couple hardware with software inextricably.
8. The symptom may be due to causes that are distributed across a number of tasks running on different processors”.

Software Testing

Induction approach

- Locate the pertinent data
- Organize the data
- Devise a hypothesis
- Prove the hypothesis

Software Testing

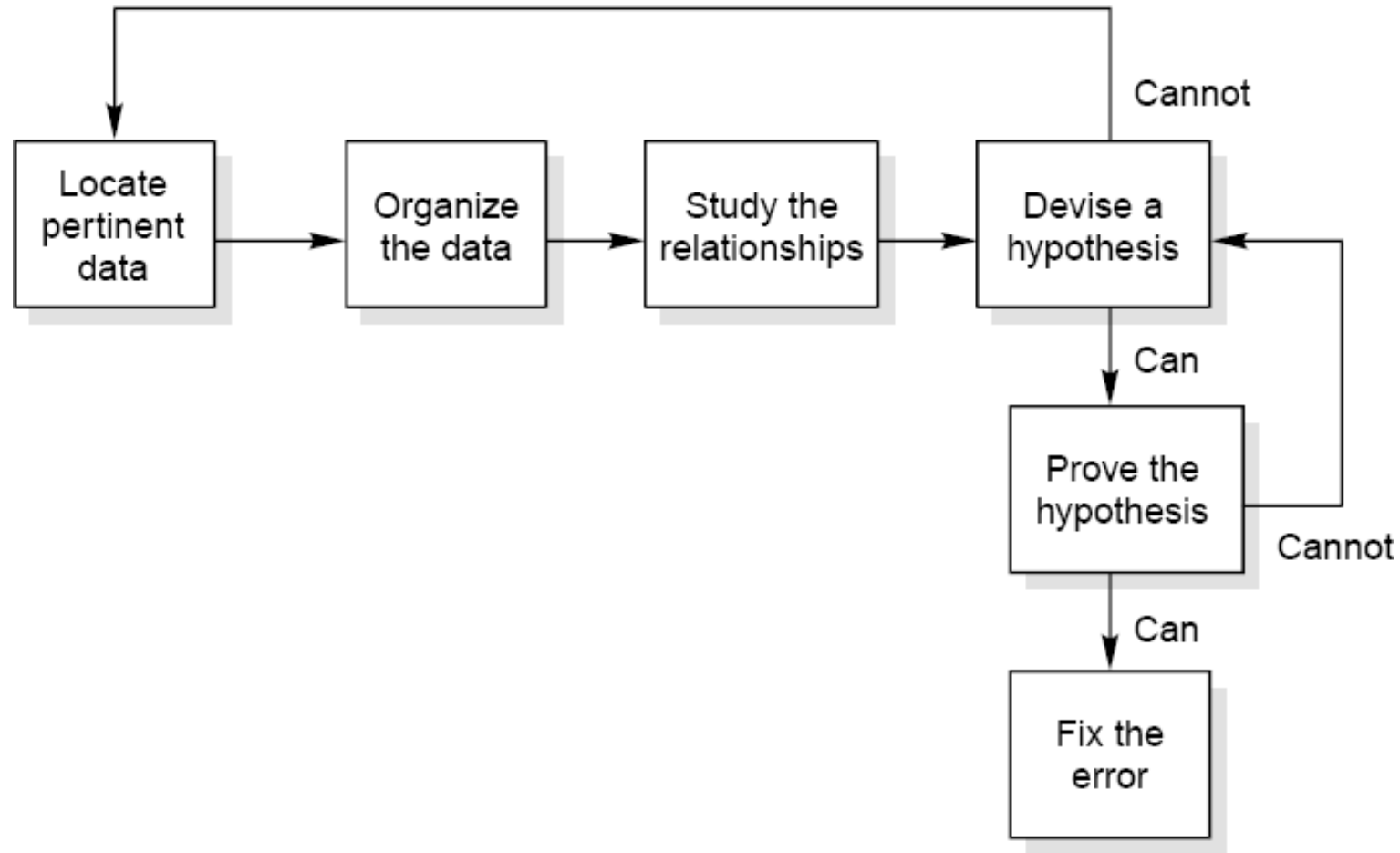


Fig. 32 : The inductive debugging process

Software Testing

Deduction approach

- Enumerate the possible causes or hypotheses
- Use the data to eliminate possible causes
- Refine the remaining hypothesis
- Prove the remaining hypothesis

Software Testing

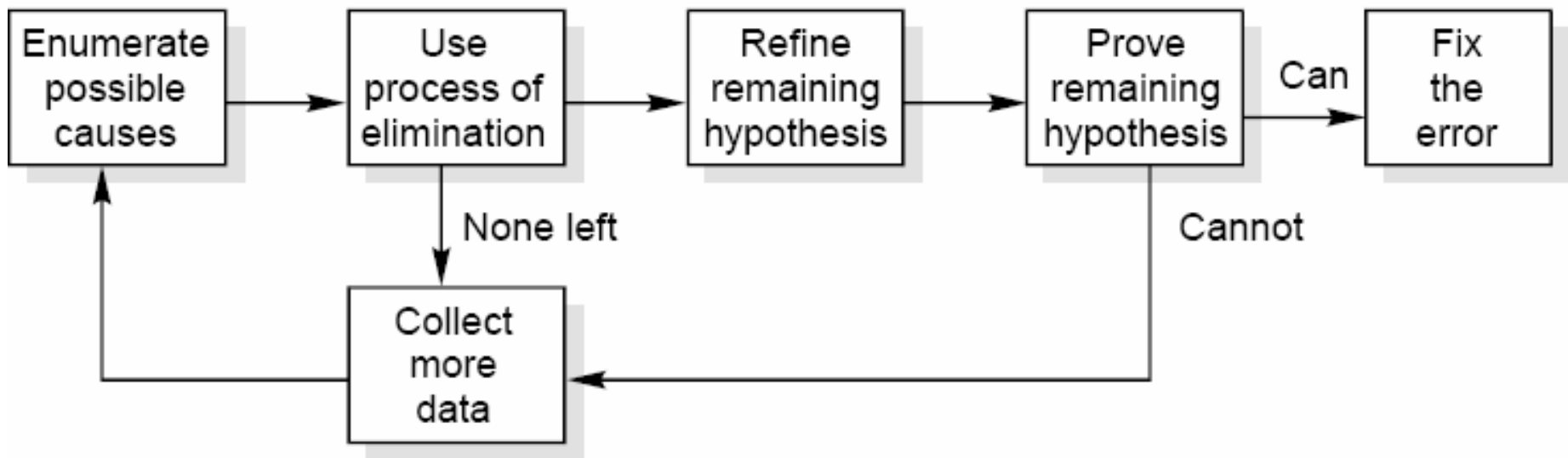


Fig. 33 : The inductive debugging process

Software Testing

Testing Tools

One way to improve the quality & quantity of testing is to make the process as pleasant as possible for the tester. This means that tools should be as concise, powerful & natural as possible.

The two broad categories of software testing tools are :

- Static
- Dynamic

Software Testing

There are different types of tools available and some are listed below:

1. Static analyzers, which examine programs systematically and automatically.
2. Code inspectors, who inspect programs automatically to make sure they adhere to minimum quality standards.
3. standards enforcers, which impose simple rules on the developer.
4. Coverage analysers, which measure the extent of coverage.
5. Output comparators, used to determine whether the output in a program is appropriate or not.

Software Testing

- 6. Test file/ data generators, used to set up test inputs.
- 7. Test harnesses, used to simplify test operations.
- 8. Test archiving systems, used to provide documentation about programs.